Machine Learning-Based Intelligent Routing Optimization for Low-Power IoT Mesh Networks

PROJECT REPORT

submitted in partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY

in

[Department / specialization to be confirmed]

SUBMITTED BY

[Student name and register number to be confirmed]

M.Tech Final-Year Project

Under the guidance of

[Project guide, designation, department to be confirmed]

[Department / Institution to be confirmed]

[Institution address to be confirmed]

Academic year: [to be confirmed]

CERTIFICATE

This is to certify that [student name], bearing register number [to be confirmed], has satisfactorily completed the project report entitled “Machine Learning-Based Intelligent Routing Optimization for Low-Power IoT Mesh Networks” in partial fulfillment of the requirements for the M.Tech Final-Year Project in [department and institution to be confirmed] during the academic year [to be confirmed].

The report presents repository-backed Python simulation outputs and a separate C++ Decision Tree application compared with the OMNeT++/INET AODV baseline. Five-seed simulator summaries are available; physical ESP32 implementation and measurements remain planned work. No physical hardware measurement or completed demonstration is certified by this report.

GUIDE	**CHAIRPERSON**
[Name, designation, department to be confirmed]	[Name, designation, department to be confirmed]

EXAMINER 1	**EXAMINER 2**

ACKNOWLEDGMENT

[Draft for author completion] I express my gratitude to [institution], [department], and [project guide] for the opportunity and guidance provided during this work. I also thank [laboratory staff, peers, and other contributors to be confirmed] for their assistance. Personalize and approve this page before submission; no names have been inferred.

[Student name]

[Register number]

Abstract

Low-power Internet of Things (IoT) networks often use intermediate nodes to deliver packets when a source cannot communicate directly with a gateway. Route quality can change as links degrade, nodes become unavailable, or traffic load changes. This project investigates machine-learning-assisted next-hop selection through a Python graph simulator, a packet-level OMNeT++/INET implementation, and a planned physical ESP32 validation study. The intended physical outcome remains a repeatable comparison against a defined routing baseline; no performance improvement is assumed in advance.

The repository contains an implemented Python graph simulator, deterministic minimum-hop and Decision Tree-assisted forwarding engines, synthetic acknowledgement-labelled observations, paired experiment code, a Streamlit interface, model artifacts, CSV results, and automated tests. It also contains a C++ MeshRoutingApp for OMNeT++/INET, generated Decision Tree inference code, AODV comparison runs, and curated five-seed packet-level results. The earlier Python raw experiment file contains 70 algorithm-level rows: five trials for each of seven scenarios and each of two modes, forming 35 paired simulated conditions. Both simulation tracks use configured or synthetic conditions; neither represents radio measurements.

The Python classifier is a depth-limited scikit-learn Decision Tree that predicts one-hop simulated delivery success from five features. Its saved evaluation uses held-out strong-link observations and records accuracy of approximately 0.961, but failure-class recall is 0.000; the confusion matrix shows that it misses every failure in that split. Separately, the INET route-choice tree was trained on 600 synthetic controlled-route rows and achieved 96.67% accuracy and 0.968 macro F1 on 120 held-out rows. These are distinct models, targets, and datasets; neither score establishes physical link prediction quality.

The repository also contains a six-node OMNeT++/INET AODV baseline and a separate C++ Decision Tree routing application. Curated paired results cover five seeds in five conditions; detailed raw run directories remain local to the simulation workspace. The planned four-board design, sensor assignments, and power architecture are documented here, but ESP32 firmware, physical packet forwarding, on-device inference, physical test data, and energy measurements are not implemented. This report distinguishes current simulation evidence from the planned hardware validation.

Keywords: low-power IoT, wireless mesh, routing, Decision Tree, packet delivery, ESP32, ESP-NOW, OMNeT++, INET.

Contents

1 Introduction....1
1.1 Context and motivation....1
1.2 IoT and low-power network characteristics....1
1.3 Mesh communication and routing....2
1.4 Problem statement....2
1.5 Aim and objectives....3
1.6 Research questions....3
1.7 Scope, contributions, and boundaries....3
1.8 Organization of the report....4
2 Literature Review and Technical Background....5
2.1 Low-power and lossy network routing....5
2.2 Conventional path-selection approaches....5
2.3 RPL and route metrics....5
2.4 Learning-assisted routing....6
2.5 Related intelligent routing studies....6
2.6 Comparative literature synthesis....7
2.7 Research gap and project positioning....7
2.8 Chapter summary....7
3 Problem Definition and Requirements....8
3.1 Formal network model....8
3.2 Routing objective and constraints....8
3.3 Functional requirements....8
3.4 Non-functional requirements....9
3.5 Hardware and software requirements....9
3.6 Assumptions and constraints....10
3.7 Success criteria and evidence levels....10
3.8 Threats to validity....11
4 Proposed Architecture and System Design....12
4.1 Architecture overview....12
4.2 Software simulation architecture....12
4.3 Proposed physical node architecture....13
4.4 Node roles and data flow....14
4.5 Packet lifecycle and control flow....15
4.6 Routing and ML interface....15
4.7 Failure, recovery, and safe fallback....15
4.8 Logging and data boundary....16
5 Hardware Design and Implementation Plan....17
5.1 Evidence status and design intent....17
5.2 Final hardware design and bill of materials....17
5.3 Node configuration....20
5.4 Radio communication and ESP-NOW boundary....21

5.5 Packet framing and interfaces....21
5.6 Firmware architecture....21
5.7 Power design and energy budget method....22
5.8 Assembly and integration plan....23
5.9 Hardware debug and acceptance approach....23
6 Routing Algorithm and Machine Learning Methodology....24
6.1 Baseline routing implementation....24
6.2 Candidate eligibility and loop control....24
6.3 Link feature construction....25
6.4 Synthetic observation generation....25
6.5 Dataset split and model training....26
6.6 Decision Tree model and target....27
6.7 Inference and candidate cost....28
6.8 Hysteresis and fallback....29
6.9 Complexity, deployment, and model risks....29
6.10 Evaluation metrics for classification....29
7 Simulation Environment and Experimental Methodology....30
7.1 Python simulation objective and topology....30
7.2 Packet process and link model....30
7.3 Experiment scenarios and pairing....30
7.4 Stored output inventory and row interpretation....31
7.5 OMNeT++/INET configuration....31
7.6 Data collection and analysis procedure....32
7.7 Reproducibility protocol....32
7.8 Simulation boundaries....32
8 Results and Analysis....34
8.1 Evidence inventory....34
8.2 Packet delivery results....34
8.3 Latency results....35
8.4 Throughput results....36
8.5 Route changes and recovery....36
8.6 Stored model evaluation....37
8.7 Interpretation by scenario....38
8.8 INET Decision Tree simulation results 39
8.9 Missing performance evidence....40
8.10 Results chapter summary....40
9 Discussion, Limitations, and Hardware Validation....41
9.1 Interpretation against objectives....41
9.2 Dataset and model limitations....41
9.3 Simulation limitations....41
9.4 Embedded and radio constraints....42
9.5 Physical validation protocol....42
9.6 Acceptance criteria and reporting....43

9.7 Risk controls....43
10 Conclusion and Future Work....44
10.1 Work completed....44
10.2 Conclusions supported by evidence....44
10.3 Remaining work....44
10.4 Future extensions....44
A Repository and Artifact Inventory....45
B Data Dictionary and Output Schemas....46
C Software Test Definition Review....48
D Routing Pseudocode and Source Mapping....49
E Simulation Configuration Reference....50
F Trial-Level Result Tables....51
G Hardware Test Protocol and Forms....53
H Reproducibility and Run Records....55
I Model Porting and Parity Checklist....56
J Outstanding Information and Submission Gate....57
K Evaluation Protocol and Statistical Interpretation....58
L Packet Format and State-Machine Design Notes....60
M Risk Register and Acceptance Gate....62
N Field Log Templates and Data Quality Rules....63
O Claim-to-Evidence Traceability Matrix....64
P Reproduction Checklist for an Independent Reviewer....65
References....66

List of Tables

Table 1.1 Project objective and evidence map....4
Table 2.1 Routing approaches in selected literature....7
Table 3.1 Functional requirements....9
Table 3.2 Non-functional requirements....9
Table 3.3 Requirements and specification status....10
Table 3.4 Assumptions and verification status....10
Table 4.1 Node roles and responsibilities....14
Table 4.2 Data and control message inventory....15
Table 5.1 Selected hardware component, quantity, role, image reference, and verification status....18
Table 5.2 Proposed frame fields; layout remains to be implemented....21
Table 5.3 Proposed firmware module boundaries....22
Table 5.4 Power measurement worksheet....23
Table 6.1 Current model feature vector....25
Table 6.2 Saved model metadata split; rows derived from run count and observations per run....27
Table 6.3 Stored synthetic classifier evaluation....28
Table 7.1 Python scenario definitions....31
Table 7.2 Saved simulation and ML artifacts....31
Table 8.1 Stored Python paired aggregate PDR results....34
Table 8.2 Five-seed OMNeT++/INET AODV and Decision Tree summary 39
Table 8.3 Result availability and missing evidence....40
Table 9.1 Planned hardware validation cases; none is claimed complete....43
Table 9.2 Project risk controls....43

List of Figures

Figure 1.1 Multi-hop delivery concept represented by the current Python graph; S is source, G gateway, and R nodes relays.....2

Figure 4.1 Repository-backed Python software architecture; arrows indicate principal calls/data flow, not separate deployed processes.....12

Figure 4.2 Proposed four-node traffic-routing topology and battery power distribution; design not assembled or electrically verified.....13

Figure 4.3 User-supplied system block diagram for the proposed four-node network and battery-powered design.....13

Figure 4.4 Current software modules and their data boundaries.....16

Figure 5.1 Representative supplier photographs of planned node, sensing, display, and indicator components 19

Figure 5.2 Representative supplier photographs of planned battery and power-distribution components 20

Figure 5.3 Physical assembly of the four-node IoT mesh prototype (photograph to be inserted after implementation).....23

Figure 5.4 Physical measurement and gateway logging arrangement (photograph to be inserted after implementation).....23

Figure 6.1 Implemented Python next-hop selection sequence reconstructed from routing/engines.py and simulator/core.py.....25

Figure 6.2 Synthetic training and evaluation workflow from ml/pipeline.py; all current rows are synthetic.....26

Figure 6.3 Saved dataset class counts by generated condition; bars are counts from data/link_observations.csv.....26

Figure 6.4 Confusion matrix from models/delivery_tree.json; held-out split contains only strong-condition runs.....28

Figure 6.5 Feature importance values from saved DecisionTreeClassifier; values describe fitted tree splits only.....28

Figure 7.1 Python graph from simulator/core.py; link attributes are synthetic/configured.....30

Figure 7.2 Six-node INET configuration layout; visual restatement of INI node coordinates, not a run screenshot.....32

Figure 8.1 Stored Python simulation PDR means with sample standard deviations; five trials per mode and scenario.....34

Figure 8.2 Mean latency for delivered packets in stored Python trials; failed packets are excluded from latency means.....35

Figure 8.3 Payload throughput means and sample standard deviations from stored Python aggregate output.....36

Figure 8.4 Stored classifier confusion matrix on held-out strong synthetic condition.....37

Figure 8.5 Stored model feature importances; values are tree split-importance statistics, not causal effects.....38

Abbreviations and Symbols

Term	Definition
ACK	Acknowledgement used to confirm a transmission outcome
AODV	Ad hoc On-Demand Distance Vector routing
CSV	Comma-separated values
ESP-NOW	Espressif connectionless Wi-Fi communication protocol
INET	OMNeT++ model library for network simulation
IoT	Internet of Things
LLN	Low-Power and Lossy Network
MAC	Medium Access Control
ML	Machine learning
OMNeT++	Discrete-event simulation framework
PDR	Packet delivery ratio
RPL	IPv6 Routing Protocol for Low-Power and Lossy Networks
RSSI	Received Signal Strength Indicator
RTT	Round-trip time
SD	Sample standard deviation
SCA	OMNeT++ scalar result file
VEC	OMNeT++ vector result file
RT	Routing-table log file
P(success)	Predicted probability that one candidate-link attempt succeeds
N	Set or count of network nodes
E	Set of modeled links
P	A route or path through the network
B	Payload size in bytes
T	Observation duration in seconds
d	Delivered packet count
n	Generated packet count

1 Introduction

1.1 Context and motivation

The Internet of Things describes systems in which physical devices sense, process, or exchange data through communication networks. An IoT deployment may include sensors, actuators, embedded controllers, local gateways, and application services. The network architecture depends on the application: a building monitor, industrial meter, agricultural sensor, or mobile field device may impose different requirements for coverage, update rate, reliability, latency, maintenance, and power. This report concerns the routing layer of a small low-power mesh concept, while avoiding assumptions about a particular sensor application because the repository does not identify a selected sensor or payload source.

Embedded IoT nodes often have less processing capacity, memory, radio airtime, and energy than conventional computers. Even when a node is powered from USB during a bench experiment, its design may still need to account for battery operation and limited duty cycle. A forwarding node also spends resources on packets generated by other devices. This makes route choice a system-level issue: choosing a path changes which nodes transmit, how many radio operations occur, how much traffic competes for airtime, and where failures appear.

A direct wireless path is not always available. Multi-hop forwarding uses an intermediate node to relay a packet towards its destination. This increases coverage but creates route dependencies: successful end-to-end transfer depends on each selected link, forwarding logic, packet format, and node availability. A route with few hops may have poor delivery conditions; a longer path may add transmissions and delay. The project explores whether multiple link observations can improve a next-hop decision in the software model, then proposes controlled physical validation.

The engineering motivation is not that machine learning is automatically superior to conventional routing. The motivation is to examine a concrete, measurable use of prediction: estimate whether one candidate-link transmission will succeed, combine that estimate with a continuation estimate, and compare the decision against a deterministic minimum-hop baseline. A useful evaluation must include failure-class model behavior, route stability, data provenance, and implementation cost alongside packet delivery.

1.2 IoT and low-power network characteristics

Low-power and lossy networks are characterized by constrained routers and interconnects. The IETF RPL specification describes limits involving processing, memory, and energy, along with links that can have high loss, low data rates, and instability [1]. These conditions differ from an ideal wired graph, where a link is often treated as persistently available and comparatively predictable. A routing method must decide how to represent a changing link, how often to measure it, and how to avoid overreacting to a single noisy sample.

An IoT application may produce periodic telemetry, event-driven alarms, commands, or bulk readings. Traffic direction and timing affect routing. Periodic small messages can tolerate a modest delay but require long-term reliability. An alarm may need fast delivery and bounded latency. A route decision cannot be evaluated independently of its traffic workload: packet size, generation rate, queueing, retry policy, measurement probes, and control traffic influence observed performance.

A mesh is a topology property, not necessarily a complete routing implementation. A radio that can exchange frames with peers does not automatically define multi-hop addressing, forwarding, route discovery, duplicate handling, or loop prevention. In the proposed hardware plan, ESP-NOW is a

candidate direct peer transport, while application firmware would provide multi-hop forwarding. That separation is important because transport capability and routing behavior are different system layers [7].

1.3 Mesh communication and routing

Model a network as a graph G=(V,E), where V is the set of nodes and E the set of modeled communication links. A route from source s to gateway g is an ordered sequence P=(v0,...,vh) with v0=s, vh=g, and each adjacent pair connected by an eligible link. The hop count is h. A route protocol determines which links are eligible, what information is collected, how alternatives are scored, and when a forwarding decision changes.

A minimum-hop policy minimizes h among available routes. It is simple and deterministic when tie-breaking is defined, but it does not directly include link success probability, measured delay, airtime cost, or energy. A metric-aware policy can combine these attributes, but each metric needs a measurement method and a normalization or cost model. Combining incomparable raw quantities without a principled scale can produce a score whose meaning is unclear.

RPL is an IPv6 routing protocol specified for low-power and lossy networks. RFC 6551 describes routing metrics and constraints that can be used for path calculation in LLNs [1], [2]. The present Python prototype is not RPL: it uses a custom graph and two routing engines. The separate INET configuration specifies AODV as a reactive baseline. These distinctions matter when interpreting both literature and implementation evidence.

$$PDR = N_delivered / N_generated;\ \ PLR = 1 - PDR \tag{1.1}$$

Here, N_delivered is the number of packets observed at the gateway and N_generated is the number originated for the same trial. Packet loss ratio (PLR) is meaningful only with the same scope and denominator. It does not by itself identify whether loss resulted from a radio error, unavailable route, hop-limit rejection, duplicate suppression, or an application timeout; logs should retain reason codes.

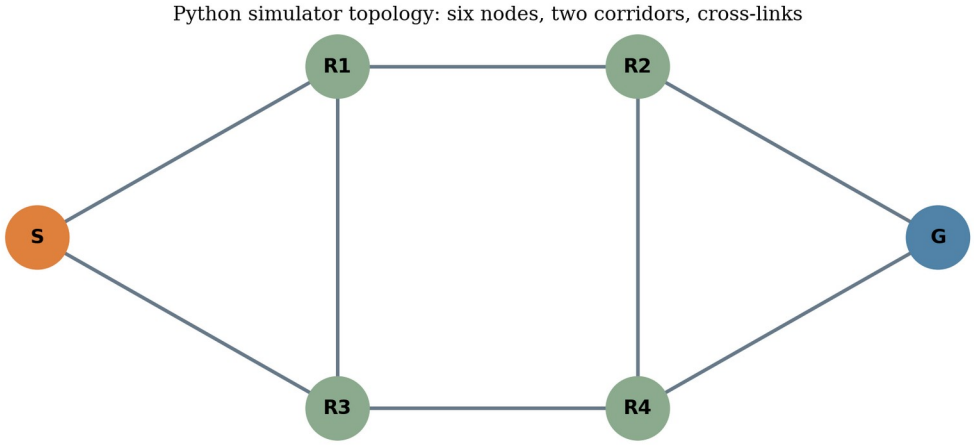


Figure 1.1: Multi-hop delivery concept represented by the current Python graph; S is source, G gateway, and R nodes relays.

1.4 Problem statement

The project problem is to investigate whether an ML-assisted next-hop selection method can use link-related features to select a route that is useful under changing synthetic conditions, and to define a credible path towards validating that method with physical nodes. The available Python simulator compares deterministic minimum-hop routing with an ML-assisted candidate scorer. Its training observations and link conditions are generated synthetically. A hardware design is proposed in the synopsis and integration note, but physical implementation is not evidenced.

The formal scope therefore has two connected but separate questions. First: what behavior does the existing Python implementation and stored CSV evidence show under its modeled scenarios? Second: what firmware, measurements, and experimental design would be needed to test a comparable policy on a small ESP32 mesh? The current work answers the first only within simulator assumptions and plans the second without claiming it has happened.

1.5 Aim and objectives

Aim: develop and evaluate a machine-learning-assisted routing design for a low-power IoT mesh, using software simulation as the present evidence stage and a physical multi-hop testbed as the planned validation stage.

- Inspect and document the repository-backed simulator, routing engines, ML pipeline, tests, outputs, and INET configuration.
- Explain a deterministic minimum-hop baseline and the current Decision Tree-assisted next-hop scoring function.
- Analyze stored paired trial outputs with correct denominators, uncertainty, and simulation boundaries.
- Identify shortcomings in saved model evaluation, especially the held-out failure-class recall and artifact seed mismatch.
- Specify a feasible hardware architecture without guessing board models, pinouts, supply ratings, sensors, or physical measurements.
- Define repeatable physical experiments that collect real link observations and compare baseline and ML modes under matched conditions.

1.6 Research questions

1. How do stored synthetic Python trial outcomes compare between minimum-hop and ML-assisted routing across the seven configured scenarios?
2. What does the saved classifier evaluation establish about success prediction and failure detection when the held-out condition is strong-link?
3. Which features and safeguards are implemented in Python, and which remain recommendations for embedded firmware?
4. What evidence, hardware integration, and experiment controls are needed to support a physical comparison on ESP32 nodes?

1.7 Scope, contributions, and boundaries

The software contribution is a small reproducible simulation design with an explicit baseline, a candidate-scoring Decision Tree policy, synthetic data generation, scenario-runner code, metrics, and stored outputs. The hardware contribution at this stage is a design and validation plan: role separation, packet fields, measurement definitions, route safeguards, and a staged test sequence. Neither contribution should be confused with a completed physical prototype or standards-compliant routing protocol.

Objective	Evidence available	Current evidence statement
Software route comparison	Python simulator, route engines, 70-row CSV; separate INET C++ app and five-seed summary	Python synthetic outputs and paired INET simulation results; full INET raw run directories remain local
Model evaluation	Joblib tree and JSON metadata	Synthetic held-out strong-condition metrics; failure recall is zero

Objective	Evidence available	Current evidence statement
Packet-level INET model	AODV baseline configuration, C++ MeshRoutingApp, generated tree, curated repeated-run summaries	Build and five-seed simulation runs recorded; no physical radio validation; AODV hop count and control overhead unmeasured
Physical mesh	Synopsis and ESP32 integration note	Design intent only; hardware, firmware, and measurements pending
Energy evaluation	No current or battery model	Not measured or estimated

Table 1.1: Project objective and evidence map

1.8 Organization of the report

Chapter 2 reviews routing concepts and related research. Chapter 3 defines the graph problem, requirements, assumptions, and success criteria. Chapter 4 explains software and proposed hardware architecture. Chapter 5 develops the hardware plan. Chapter 6 explains implemented routing and ML. Chapter 7 documents simulation and experiment methods. Chapter 8 analyzes stored results. Chapter 9 discusses limits and physical validation. Chapter 10 summarizes supported conclusions and next work. Appendices map the repository, schemas, test definitions, configurations, trial values, and planned protocols.

2 Literature Review and Technical Background

2.1 Low-power and lossy network routing

RFC 6550 defines RPL for low-power and lossy networks and describes multipoint-to-point, point-to-multipoint, and point-to-point traffic patterns [1]. Its context is relevant because constrained routers and variable links complicate path selection. The standard establishes protocol mechanisms and terminology; it does not imply that the present project implements RPL. The project's Python network is a custom graph and its INET stage is separately configured around AODV.

RFC 6551 discusses metrics and constraints that can support path calculation in LLNs, including link and node properties [2]. This provides a standards-based reference for why path evaluation may consider more than hop count. Metrics still require operational definitions: how they are observed, refreshed, combined, and carried between nodes. A metric can become stale between measurement and forwarding, and measurement traffic itself consumes airtime and energy.

In a constrained mesh, route choice is coupled to control overhead. Frequent route updates may react quickly but create traffic and processing cost; slow updates reduce overhead but can leave nodes using stale paths. The project plan includes periodic neighbor beacons and sliding-window link estimates, but those mechanisms are proposed, not implemented in firmware. A physical experiment must set beacon cadence and record its overhead before making claims about energy or channel use.

2.2 Conventional path-selection approaches

Shortest-path methods are a useful reference because their objective is explicit. In the Python baseline, available links form the graph supplied to the shortest-path routine; among equal-length paths, the lexicographically smallest path is selected. This creates repeatability but embeds a particular tie policy. It is not an implementation of a full distributed link-state protocol: nodes share a simulator graph and the simulation does not charge airtime for route discovery.

Reactive routing establishes or repairs routes in response to demand or topology change. AODV is the INET baseline comparison. Its behavior is assessed through packet-level delivery traces; the separate Decision Tree application logs selection and route events. Curated repeated-run summaries report five matched seeds across five conditions. Full raw run directories remain local, and the recorded results do not establish behavior on physical hardware.

RPL organizes routing for LLN requirements and may use an objective function to rank candidate parents. RPL and AODV differ in routing model, control behavior, and implementation context. Therefore the project should report Python minimum-hop, INET AODV, and any eventual hardware baseline under distinct labels unless a precise equivalence is demonstrated.

2.3 RPL and route metrics

An objective function maps one or more metrics into a route-selection preference. A metric may be additive, multiplicative, constrained by a threshold, or used as a tie-breaker. ETX-like expectations, hop count, latency, residual energy, and reliability have different units and error characteristics. RFC 6551 provides an interoperable vocabulary for routing metric objects and constraints [2], but a student implementation still needs a local measurement specification.

RSSI represents received signal strength at a receiver. It can help characterize received frames but is not a direct probability of future delivery. A reading varies with placement, antenna, orientation, interference, channel, and hardware. A project-specific model may learn associations from RSSI and other features, but those associations require representative labeled observations. Synthetic RSSI values cannot establish the radio behavior of a selected ESP32 board.

Latency must specify endpoints and clock basis. A send-to-ACK interval is round-trip time unless clocks are synchronized and one-way timing is derived using a valid method. The Python model uses configured link latency as simulated per-hop delay and sums it over a delivered path. It does not implement packet-level clock synchronization or live radio timing.

Loss estimation should specify denominator, measurement window, and ACK timeout. If an observation window contains m attempts and a successful response is received for a attempts, a simple empirical success estimate is a/m. Short windows react quickly but have high variance; long windows smooth noise but react slowly. This trade-off is a design consideration for planned hardware measurement, not a feature tested on the current physical network.

2.4 Learning-assisted routing

ML-RPL integrates a CatBoost-based model with RPL parent selection for wireless smart-grid networks [3]. Its central idea is to predict a delivery-related quantity from routing metrics and use it in parent selection. That is conceptually related to this project, but the model, features, protocol, topology, and evidence differ. The present prototype uses a scikit-learn Decision Tree on synthetic generated observations and does not implement RPL or CatBoost.

PEARL explores a Q-learning routing policy that considers delay and power for IoT applications [4]. Its objective and learning paradigm differ from a supervised classifier predicting one-hop success. Because the exact bibliographic venue and pagination of the supplied PDF were not independently verified, the reference is marked for confirmation in the bibliography and is used only as a qualitative research lead, not as an authority for a numerical claim.

A 2023 review surveys ML methods and open challenges for intelligent routing [5]. It is useful for framing data quality, model selection, inference placement, and generalization as research challenges rather than guaranteed benefits. An ML model trained on one topology or condition can fail under a distribution shift. Held-out runs or conditions provide stronger separation than random row-level partitioning, although only physical data can test radio-domain transfer.

A 2026 Sensors study proposes GraphSAGE-MAPPO for multi-layer dynamic mesh routing and resource allocation [6]. Its scale and model complexity are substantially different from the current shallow Decision Tree prototype. It illustrates a broader family of graph representation and multi-agent learning approaches, but is not a direct baseline for the small fixed graph used here.

2.5 Related intelligent routing studies

Lloret and colleagues describe a group-based AI routing approach for M2M IoT networks that uses an evolutionary multi-objective technique [7]. Its gateway-oriented service discovery and resource criteria are distinct from per-packet next-hop forwarding in this repository. It contributes background on how routing objectives can be application-specific, not evidence that its reported behavior transfers to this implementation.

The literature reflects a spectrum of design complexity. Rule-based costs are transparent and compact. Supervised learning predicts labels represented in a dataset. Reinforcement learning adapts policies from reward and interaction. Graph neural networks encode relationships among nodes and links. Complexity and data needs rise across these choices, but the choice must match deployment constraints and evaluation resources.

The project’s Decision Tree was selected for interpretability and potential threshold-based export. That rationale is a design choice documented in the synopsis, not proof that the tree fits embedded memory or executes within a useful time on the eventual board. Feature importances stored with the fitted tree are relative split criteria, not causal explanations and not measurements of network contribution.

2.6 Comparative literature synthesis

Approach	Main idea	Advantages reported or inherent	Limitations / relevance here
Minimum-hop baseline	Select shortest available path	Simple objective; easy to reproduce	Ignores quality except availability; implemented in Python
RPL [1]	IPv6 routing protocol for LLNs	Designed for constrained LLN traffic patterns	Not implemented in this project
LLN metrics [2]	Standard metric/constraint definitions	Helps describe path criteria consistently	Does not determine local measurement quality
ML-RPL [3]	CatBoost probability-based RPL parent choice	Related learned link-quality approach	Different protocol/model; no direct reproduction
PEARL [4]	Q-learning with delay/power objective	Adaptive policy concept	Citation details need confirmation; not baseline
Review [5]	Survey of ML routing methods/challenges	Context on model and deployment trade-offs	Broad survey; not project performance evidence
GraphSAGE-MAPPO [6]	Graph features with multi-agent policy learning	Handles graph context and joint decisions	Much more complex than present small testbed
M2M group-based AI [7]	Evolutionary multi-objective gateway selection	Application-aware service/resource selection	Different problem formulation and architecture

Table 2.1: Routing approaches in selected literature

2.7 Research gap and project positioning

The project gap is practical and bounded: determine whether a small supervised link-success estimator can be integrated into a multi-hop forwarding design with safeguards, first in a software simulator and later in a physical testbed. The current evidence supports code-level exploration but leaves real data, firmware costs, and physical repeatability unresolved.

The project is positioned between a deterministic shortest-path policy and more elaborate learning-based protocols. It can evaluate candidate scoring and provide interpretable decisions. It cannot claim standards compliance, large-scale generalization, or energy optimization from available files. Its contribution will depend on completing physical forwarding and reporting transparent paired measurements.

2.8 Chapter summary

The literature establishes relevant LLN routing concepts and multiple intelligent routing paradigms. It does not validate the project's model or hardware. This report therefore uses standards and papers to frame technical choices, while treating repository code and data as the evidence for project-specific claims.

3 Problem Definition and Requirements

3.1 Formal network model

Let G=(V,E) denote a graph with source s, gateway g, and relay vertices. Each edge e=(u,v) has a state containing availability and link attributes. The Python simulator models attributes as available, RSSI, loss probability, latency, capacity, and congestion. Some values are configured globally; scenarios modify them. The data generator constructs separate synthetic feature observations. Neither representation is calibrated to hardware.

A candidate route P=(v0,...,vh) begins at source s=v0 and ends at gateway g=vh. Each transition (vi,vi+1) must be an edge that is available to the forwarding logic. A packet carries a history of visited vertices and a hop limit. A packet is delivered if its processing reaches the gateway before a terminal event such as route unavailability, hop-limit rejection, or link loss.

$$G = (V,E);\ P(s,g) = (v_0{=}s, v_1, \ldots, v_h{=}g),\ (v_i,v_{i+1}) \in E \quad (3.1)$$

3.2 Routing objective and constraints

The baseline objective is to minimize hop count over available edges. The ML engine instead scores valid candidate neighbors. For candidate v, q(v) is the Decision Tree probability for next-hop success. Ptail(v) is a product of configured link-success probabilities along a shortest-hop continuation. The score is C(v)=(1−q(v))+(1−Ptail(v)). This expression adds two bounded risk terms; it is a project-specific heuristic, not a calibrated expected end-to-end loss probability.

Eligibility constraints exclude a neighbor already in the packet history, an unavailable link, a neighbor with no route continuation, or a continuation that returns through the current node. The packet hop limit supplies another bound. When no candidate can be scored, Python falls back to minimum-hop. When the previous next hop remains eligible, the 0.08 margin may retain it if its cost is within that margin of the best cost.

No energy term appears in the implemented score. The architecture motivation includes low-power operation, but the current code neither measures current nor estimates battery depletion. Likewise, the score does not include queue depth, routing-control overhead, or measured channel occupancy. Future revisions must not describe those quantities as optimized until they are added with a defined measurement model and validated.

$$C(v) = [1 - q(v)] + [1 - P_tail(v)];\ v^* = \arg\min_v C(v) \quad (3.2)$$

3.3 Functional requirements

ID	Requirement	Trace / status
FR-01	Originate packets with unique source-scoped identifiers	Python packet ID exists; embedded source format planned
FR-02	Forward packets over eligible next hops	Python graph forwarding exists; firmware pending
FR-03	Provide minimum-hop baseline mode	Implemented in Python
FR-04	Provide model-assisted candidate scoring	Implemented in Python using stored/fitted classifier
FR-05	Reject route loops and hop-limit exhaustion	Simulator checks; bounded embedded duplicate cache planned
FR-06	Track delivered and dropped packets with reason	Python records status; gateway logging planned
FR-07	Compare both modes with paired conditions	Experiment runner and stored raw outputs exist
FR-08	Collect link measurements from physical nodes	Not implemented; planned

ID	Requirement	Trace / status
FR-09	Support fallback when model/candidates fail	Python baseline fallback exists; invalid-feature behavior requires hardware implementation
FR-10	Persist repeatable experimental artifacts	CSV outputs exist; formal run manifest/checksum process pending

Table 3.1: Functional requirements

3.4 Non-functional requirements

ID	Property	Acceptance direction
NFR-01	Determinism	Fixed seeds should reproduce the same synthetic inputs and run records.
NFR-02	Safety of forwarding	Malformed, duplicate, looping, or exhausted packets should not be forwarded.
NFR-03	Resource suitability	Embedded RAM, flash, inference time, callback latency, and power must be measured.
NFR-04	Measurement traceability	Every physical observation must retain unit, timestamp, node, link, window, and trial ID.
NFR-05	Comparability	Baseline and ML trials must use matched topology, workload, and environmental setup.
NFR-06	Portability	Selected board and firmware version must be fixed before implementation claims.
NFR-07	Interpretability	Decision outputs should log candidate score, model probability, and fallback reason.
NFR-08	Evidence integrity	Synthetic, simulator, and physical evidence must be labeled separately.

Table 3.2: Non-functional requirements

3.5 Hardware and software requirements

The final intended physical design uses four ESP32 DevKit V1 boards with 38-pin headers: Node 1 is the source, Nodes 2 and 3 are alternative relays, and Node 4 is the destination. Sensors are installed only at Node 1: one DHT11, one LDR, and one sound sensor. Node 1 also has one 1.3-inch, 128×64 I²C OLED. Each node has one traffic-light indicator module. Sensor/module SKUs, pin assignments, board revisions, and electrical interfaces remain to be verified; this specification describes the intended build, not purchased or assembled hardware.

The Python software lists Python 3.11+, NetworkX, NumPy, pandas, scikit-learn, Streamlit, Plotly, and tests. The repository requirements file should be treated as the install authority for an actual environment. OMNeT++/INET are a separate simulator toolchain with pinned proposed versions recorded in the INET README, but not installed in the inspected environment.

Category	Requirement / proposal	Evidence boundary
Embedded boards	Four ESP32 DevKit V1 boards with 38-pin headers	Intended design; exact revision/procurement not verified
Node roles	Node 1 source; Nodes 2/3 relay alternatives; Node 4 destination	Planned topology; firmware and radio behavior unimplemented

Category	Requirement / proposal	Evidence boundary
Sensing/display	Node 1: one DHT11, one LDR, one sound sensor, one 1.3-inch 128×64 I²C OLED	Component assignments set; pinout and electrical interface unverified
Indicators	One traffic-light indicator module per node; four total	Module voltage/current and driver requirements unverified
Power	4S 18650 4 × 2000 mAh pack, 4S BMS, switch, four separate 5 V buck rails	Design calculations only; wiring, ratings, and commissioning pending
Radio	ESP-NOW candidate direct peer transport with application-level forwarding	No firmware or physical forwarding evidence
Backend/logger	Node 4 destination with planned logging/backend export	Interface and implementation not present

Table 3.3: Requirements and specification status

3.6 Assumptions and constraints

The Python graph contains at least six nodes and alternate routes. The simulation processes packets sequentially and uses seeded per-link Bernoulli delivery draws. Its latency is synthetic and deterministic in the stored configurations. These are model assumptions rather than claims about a deployed mesh.

The parent INET configuration defines six stationary nodes and a unit-disk radio model with configured communication range. It provides an AODV baseline and UDP source/sink traffic. The separate ML routing project adds a C++ Decision Tree application; local build and repeated simulation runs are recorded. The INET radio abstraction is not calibrated ESP-NOW behavior.

The hardware plan assumes that the selected ESP32 and API expose the required peer send/receive behavior and some usable RSSI information. Those assumptions need board/version confirmation. Latency will initially be measured via probe/ACK round trips unless clock synchronization is implemented. Throughput and loss require explicit windows and counts.

Assumption	**Consequence if false**	**Verification**
At least two viable paths exist	Route alternatives and recovery unavailable	Topology connectivity check; physical peer link test
ACK or probe identifies one-hop outcome	Supervised label ambiguous	Define timeout and correlate request/ACK IDs
RSSI is exposed by chosen API	Feature missing on target	Check callback/driver documentation and log test frames
Model input units match training	Prediction invalid under deployment drift	Schema/unit assertions and parity vectors
Paired seeds represent same simulation conditions	Comparison no longer paired	Join raw baseline/ML rows on scenario and seed

Table 3.4: Assumptions and verification status

3.7 Success criteria and evidence levels

Software success means that configured code paths execute as designed under documented simulator assumptions and output artifacts can be traced to commands and source revision. Test definitions

provide intended checks, but this report does not claim a fresh test run. Stored outputs show that records exist, not necessarily that the current repository state generated them.

INET success requires a pinned environment, successful build, a Normal run with gateway reception, visible forwarding through relay nodes, scenario traces, raw SCA/VEC/RT retention, and exporter agreement. Configuration files alone do not satisfy these criteria.

Physical success requires board inventory, peer connectivity, fixed-route multi-hop forwarding, valid packet accounting, controlled failure/recovery tests, and a paired baseline/ML experiment. ML success should be defined with class-sensitive metrics and practical costs, not overall accuracy alone. Energy success requires a defined current measurement and should not be inferred from packet counts.

3.8 Threats to validity

- Construct validity: simulated features and labels may not correspond to physical link quality.
- Internal validity: random loss, common random numbers, scenario transformations, and route tie-breaking shape outcomes.
- External validity: six-node synthetic topology cannot establish performance in larger or mobile deployments.
- Statistical validity: five paired trials per scenario yield descriptive evidence; they do not justify broad significance claims.
- Measurement validity: latency, RSSI, loss, throughput, route switches, and energy need explicit operational definitions.
- Reproducibility validity: model seed 3 and trial seeds beginning at 17 require artifact provenance reconciliation.

4 Proposed Architecture and System Design

4.1 Architecture overview

The architecture has two implemented software paths and a planned physical path. The Python path begins with graph topology and link attributes, simulates packet generation and forwarding, invokes a routing engine, samples delivery outcome, and accumulates packet records for metric calculation. Its ML pipeline generates synthetic training rows, fits a classifier, and provides predictions to the scorer. The separate OMNeT++/INET path runs a C++ MeshRoutingApp with generated Decision Tree inference beside an AODV baseline. The experiment runner pairs modes and writes raw and aggregate results. The physical ESP32 path remains planned.

The intended physical path begins at a source node, crosses one or more relay nodes, reaches a gateway, and sends logs to a laptop or equivalent collector. Radio callbacks should receive/transmit frames; a separate routing task should validate and forward packets. Feature sampling and model inference belong above the transport API. This avoids blocking high-priority radio callbacks with routing or ML work, consistent with Espressif guidance to keep callback work short and defer processing [7].

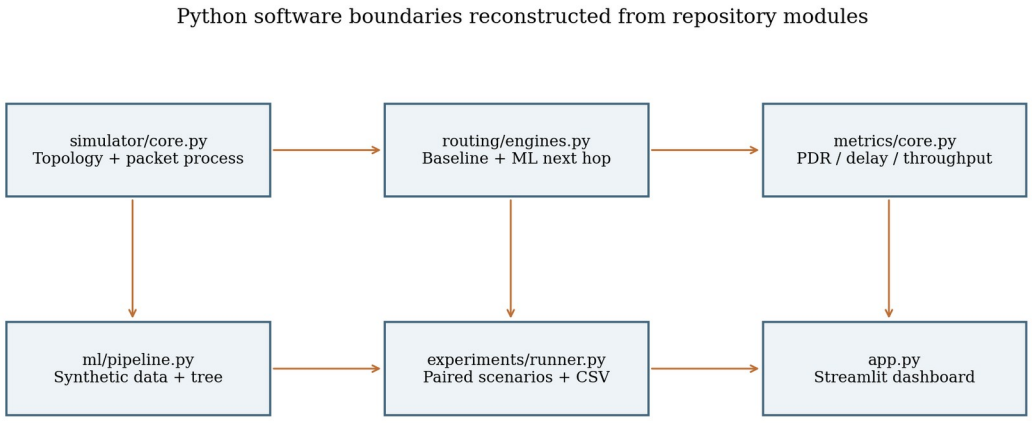


Figure 4.1: Repository-backed Python software architecture; arrows indicate principal calls/data flow, not separate deployed processes.

4.2 Software simulation architecture

`simulator/core.py` owns graph construction, packet/config dataclasses, scenario state, link-feature transformation, packet loop, link outcome draw, path history, and helper for duplicate suppression. `routing/engines.py` implements deterministic minimum-hop and ML candidate scoring. `ml/pipeline.py` creates synthetic observations, trains/evaluates the Decision Tree, writes CSV/joblib/JSON artifacts, and loads the model. `metrics/core.py` calculates PDR, loss, latency, throughput, and packet counts. `experiments/runner.py` executes paired scenario trials and writes output files.

`app.py` exposes a Streamlit dashboard over the same simulation and experiment APIs. It allows configuration of node count, packet workload, link values, traffic load, seed, scenario, and route mode; shows topology and packet events; and provides CSV downloads. UI controls do not demonstrate that a user session was executed or that outputs are hardware measurements.

The module boundary separates model training, inference, routing, packet simulation, metrics, and presentation. The INET application adds packet-level forwarding, candidate validation, route metadata, duplicate suppression, loop checks, and hop-limit enforcement within the simulator. Neither software

path implements ESP32 firmware task scheduling, real radio peers, measured cross-node neighbor state exchange, or a deployed route-cost advertisement protocol.

4.3 Proposed physical node architecture

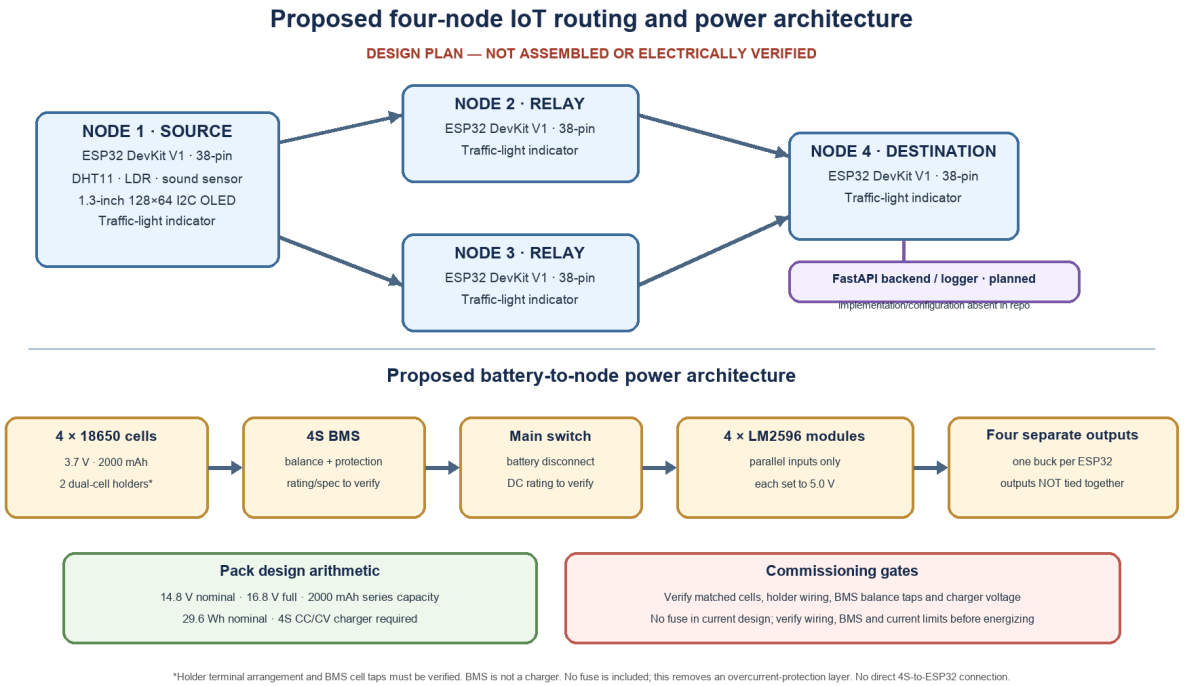


Figure 4.2: Proposed four-node traffic-routing topology and battery power distribution; design not assembled or electrically verified.

The supplied system diagram summarizes planned sensor assignments, node roles, backend flow, and power distribution. Component models, ratings, interfaces, and wiring remain unverified; diagram is not a physical board-placement drawing.

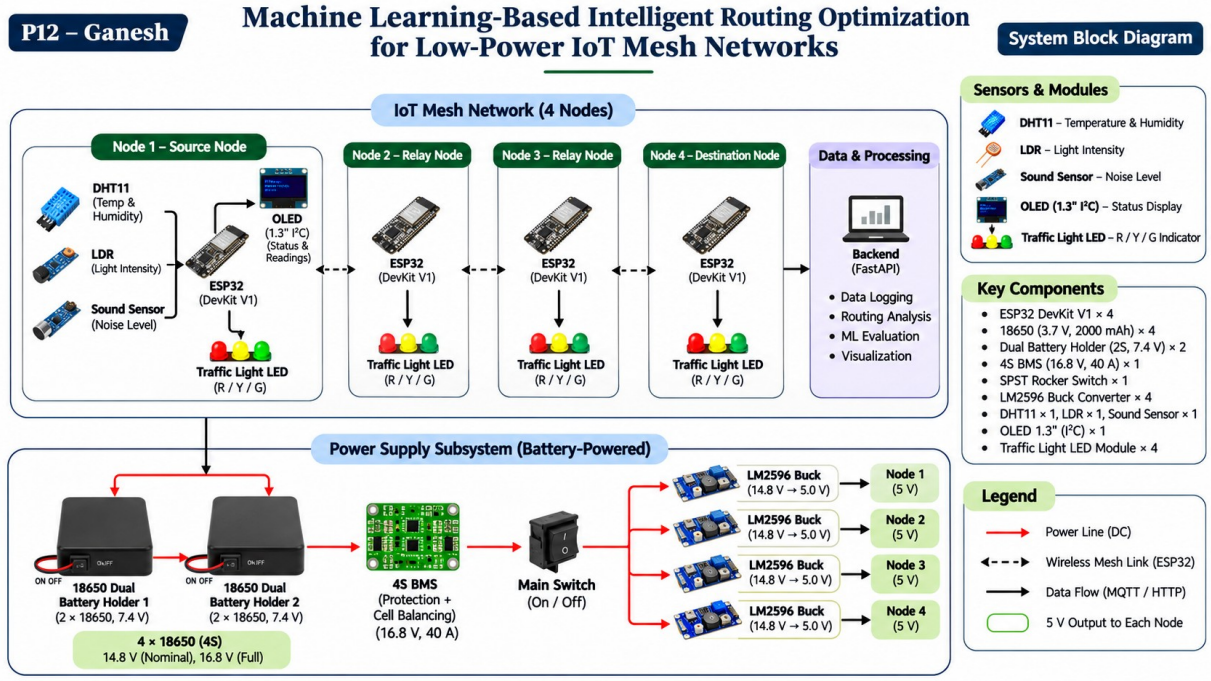


Figure 4.3: User-supplied system block diagram for the proposed four-node network and battery-powered design; not assembled or electrically verified.

The planned physical route alternatives are Node 1 → Node 2 → Node 4 and Node 1 → Node 3 → Node 4. Node 1 alone samples the DHT11, LDR, and sound sensor and presents local information on the OLED; all four nodes have a traffic-light indicator. Nodes 2 and 3 relay application packets. Node 4 terminates them. ESP-NOW is the proposed peer transport; multi-hop forwarding and route selection remain application responsibilities. Node 4 is intended to export records to the FastAPI backend shown in the original project diagram. No backend implementation or configured interface is present in the inspected repository; transport, schema, and successful delivery remain unverified.

A four-board physical network can compare the two relay corridors under controlled conditions, subject to both routes being independently reachable. The Python simulator's default six-node graph and the separate six-router INET/AODV configuration are software experiments; neither is a claim that six physical nodes exist. No node other than Node 1 is assigned sensing hardware in this final design.

4.4 Node roles and data flow

Role	Inputs	Processing responsibilities	Outputs
Node 1 — source/sensing	DHT11, LDR, sound sensor; I²C OLED; traffic-light indicator	Sample local inputs, display status, assign packet ID, originate payload	ESP-NOW frame toward Node 2 or Node 3

Role	Inputs	Processing responsibilities	Outputs
Node 2 — relay	Peer frame; local neighbor/metric state; traffic-light indicator	Validate, suppress duplicates, select/forward eligible next hop	Forwarded frame toward Node 4 or logged drop
Node 3 — relay	Peer frame; local neighbor/metric state; traffic-light indicator	Same relay safeguards as Node 2; alternate corridor	Forwarded frame toward Node 4 or logged drop
Node 4 — destination	Incoming frame; traffic-light indicator; local receive clock	Validate destination, account packets, record receive/path data	Local log and planned backend/logger export

Table 4.1: Node roles and responsibilities

Control information differs from application data. A beacon may advertise node identity and neighbor availability. Probe/ACK messages provide observations for link success, RTT, and loss. Data frames carry application payload and per-packet identity. Route-cost advertisements, if used, need a versioned format and freshness policy. No such embedded message protocol currently exists, so the inventory below is a proposed design surface rather than implemented behavior.

Message class	Purpose	Minimum fields to define	Current status
Discovery beacon	Neighbor presence	node ID, protocol version, sequence/time	Planned
Probe	Sample candidate link	probe ID, origin, send time, payload class	Planned
ACK	Associate outcome with attempt	probe/data ID, responding peer, status	Synthetic label only in Python
Application data	Source-to-gateway payload	source, destination, ID, hop limit, payload	Python packet object; firmware pending
Route update	Optional continuation cost	origin, destination, cost, freshness	Not implemented

Table 4.2: Data and control message inventory

4.5 Packet lifecycle and control flow

A packet lifecycle begins when a source application requests transmission. The source allocates an identifier, records creation time, sets destination and hop limit, and encodes the payload. On receipt, a node checks the version, frame size, destination, identifier, hop limit, visited path, and duplicate cache. A gateway consumes packets addressed to it; a relay selects an eligible neighbor and forwards.

At each relay, a route decision should use a consistent snapshot of neighbor state. If feature samples are stale, missing, or outside expected bounds, the policy should choose a safe baseline or reject the candidate. The routing result should include a reason code so later analysis can distinguish ML selection, hysteresis retention, baseline fallback, unavailable route, and invalid input.

The project synopsis proposes packet sequence numbers, timestamps, hop count, hop limit, and route/path information. The integration plan adds protocol version and payload length, bounded duplicate suppression, and malformed-frame rejection. The exact byte layout is not specified and must be selected after target and payload constraints are known.

4.6 Routing and ML interface

In the Python implementation, the simulator calls a routing engine at each forwarding node. The engine receives the graph, current node, destination, packet history, model, load, and previous next hop. The resulting neighbor and reason string guide the next simulated transmission. This interface makes route logic testable apart from packet outcome simulation.

For ESP32 integration, a narrow route interface should accept candidate-neighbor records and return a chosen neighbor plus explanation. The interface should use explicit units, validity flags, and a versioned schema, without exposing dynamic Python objects to firmware. A generated tree evaluator could consume a fixed-size array after validation. Generated C++ inference and parity tests exist for the INET simulation; an ESP32-specific evaluator and feature parity with physical measurements have not been implemented or tested.

4.7 Failure, recovery, and safe fallback

Current Python failure conditions include route unavailable, hop limit, loop prevention, duplicate suppression, link unavailable, and packet loss. The recovery scenario makes a preferred source link unavailable for the first half of packet IDs and restores it at the midpoint. It measures time from the modeled recovery point to first delivered packet in the post-recovery half. This is a discrete state toggle, not a measured radio recovery process.

In hardware, a relay may stop transmitting, lose power, move, or become temporarily unreachable. The INET application handles simulated node/link state, validates candidates, and requires an end-to-end request/return probe before a restarted relay becomes eligible; the ESP32 implementation still lacks neighbor timeout, ACK retry, and physical route recovery. A test should distinguish detection delay from route recomputation delay and end-to-end delivery delay. Fallback must not route through an unavailable or loop-forming neighbor even if the ML model assigns a high probability.

4.8 Logging and data boundary

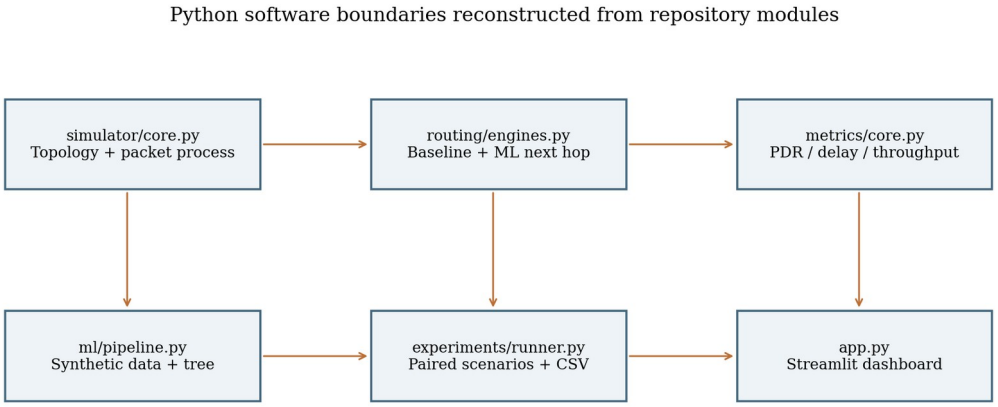


Figure 4.4: Current software modules and their data boundaries.

The software experiment outputs include packet-level results and aggregate metrics. A physical data pipeline should preserve raw observations before transformations, record model version and trial configuration, and avoid overwriting the original synthetic dataset. Raw data should include enough fields to reconstruct PDR, latency, throughput, route-switch count, and model outcomes. When an output value is missing, record why rather than substituting zero.

5 Hardware Design and Implementation Plan

5.1 Evidence status and design intent

The repository contains no assembled prototype, ESP32 firmware, physical measurement logs, verified pin map, or purchasing evidence. The separate OMNeT++/INET C++ routing application is simulation software and does not constitute ESP32 firmware. The intended hardware design specifies four ESP32 DevKit V1 38-pin boards with one source, two relay alternatives, and one destination. This chapter documents the proposed BOM and safe commissioning sequence. It does not claim that parts have been procured, wired, or tested. The current project files contain no approved wooden-board layout image; Figures 4.2–4.3 are network and power architecture diagrams, not physical placement drawings.

The proposed physical topology is Node 1 → Node 2 → Node 4 and Node 1 → Node 3 → Node 4. Each path contains one relay. Compare them only after confirming both links and the destination are reachable; controlled attenuation or relay/link disablement can then test route selection and recovery.

5.2 Final hardware design and bill of materials

Table 5.1 records intended components, quantities, image references, and evidence status. Figures 5.1–5.2 show genuine supplier product photographs as representative references only; they do not document purchased or assembled project hardware. Exact board revision, sensor interfaces, traffic-light electrical requirements, holder wiring, BMS data sheet, and charger compatibility remain open. The proposed 4S design has not been assembled or electrically verified.

Component	Role / specification	Quantity	Status / verification	Photo ref.
ESP32 DevKit V1, 38-pin	Source Node 1; relay Nodes 2/3; destination Node 4	4	Intended model; confirm exact board/revision and supply interface	Fig. 5.1
DHT11	Temperature/humidity input at Node 1 only	1	Sensor assignment set; exact module voltage/interface unverified	Fig. 5.1
LDR module/sensor	Light input at Node 1 only	1	Confirm analog range, divider/module circuit, and ADC pin	Fig. 5.1
Sound sensor	Sound-level/event input at Node 1 only	1	Confirm module/output type, safe voltage, and signal meaning	Fig. 5.1
I²C OLED, 1.3 in, 128×64	Node 1 display	1	Confirm controller, address, voltage, and I²C levels	Fig. 5.1
Traffic-light indicator module	One indicator module per node	4	Confirm module voltage/current and required drivers	Fig. 5.1

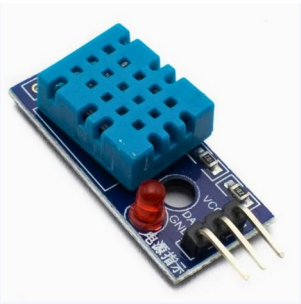
Component	Role / specification	Quantity	Status / verification	Photo ref.
18650 Li-ion cell, 3.7 V, 2000 mAh	Four-cell 4S pack; matched cells required	4	Specified intent; cell make/condition and matching unverified	Fig. 5.2
Dual-cell holder	Two holders, intended 2S series segments	2	Terminal arrangement and series wiring must be verified	Fig. 5.2
4S balancing/protection BMS	Pack protection/balancing	1	Listing advertises 40 A; verify exact rating/pinout and ambiguous 16.8–18.1 V charge spec	Fig. 5.2
Main switch	Disconnect protected pack output	1	Current/DC rating and pole arrangement unselected	Fig. 5.2
LM2596 buck converter module	One 5.0 V rail per ESP32; inputs parallel, outputs separate	4	Set/measure each; generic module continuous rating/thermal behavior unverified	Fig. 5.2
4S Li-ion CC/CV charger	Charge 4S pack to chemistry-appropriate 16.8 V termination	1	Exact compatible charger unselected; BMS is not a charger	—

Table 5.1: Selected hardware component, quantity, role, image reference, and verification status



ESP32 DevKit V1 · 38-pin

Photo: Robocraze [C1]; representative



DHT11 temperature / humidity

Photo: Robocraze [C2]; representative



LDR light-sensor module

Photo: Robocraze [C3]; representative



Microphone sensor · analog/digital listing

Photo: Robocraze [C4]; representative



1.3-inch 128×64 I²C OLED

Photo: Robocraze [C5]; representative



Red / yellow / green indicator

Photo: REES52 [C6]; representative

Figure 5.1: Representative supplier photographs of planned node, sensing, display, and indicator components.

Product photographs credited by supplier code in reports/component_photos/ATTRIBUTION.md. Images identify component types; exact purchased units and hardware behavior remain unverified.



18650 Li-ion cell · listing 3.7 V / 2000 mAh

Photo: Robocraze [C7]; representative



Two-cell holder · one holder shown

Photo: Robocraze [C8]; representative



4S 40A balancing BMS reference

Photo: HIT.PS [C9]; representative



LM2596 buck converter module

Photo: Robocraze [C10]; representative



SPST rocker · listed 12–24 V DC

Photo: Spider Automation [C11]; representative



Male-to-female jumper leads

Photo: Robocraze [C12]; representative

Figure 5.2: Representative supplier photographs of planned battery and power-distribution components.

Product photographs credited by supplier code in reports/component_photos/ATTRIBUTION.md. Images identify component types; exact purchased units and hardware behavior remain unverified.

5.3 Node configuration

Node 1 is the only sensing node. It reads one DHT11, one LDR, and one sound sensor, presents local status on a single 1.3-inch 128×64 I²C OLED, and originates packets. Sensors require documented module voltage, output type, signal conditioning, and pin assignments before firmware work; do not assume analog/digital compatibility from component names alone. Node 1 also has one traffic-light indicator. Nodes 2 and 3 relay packets and each have one indicator; Node 4 receives packets and has one indicator. No additional sensor is assigned to those three nodes.

The Node 1 OLED is intended to display sampled temperature, humidity, ambient-light and sound-event readings together with packet/route status. Each traffic-light module is intended to show green for a healthy network state, yellow during transmission or forwarding, and red for sustained communication failure. These meanings are firmware plans only; no sensor readout, display, LED, or routing behavior has been tested on hardware. The sound module is a sound-event/intensity proxy and is not a calibrated sound-pressure-level meter.

Relay nodes should run the same forwarding framework with role-specific configuration. A relay needs neighbor state, duplicate tracking, packet validation, route policy, and a transmission queue. A gateway should share frame validation and duplicate handling while adding sink accounting and logging. Avoid assuming that a logical gateway must run ML; the ML placement remains an implementation option, not a current fact.

Node configuration must retain firmware version, board revision, radio channel, transmit power setting if configurable, peer list, retry policy, beacon interval, and application workload. These variables affect range and channel behavior. The project has not selected these values, so the physical test protocol should log them rather than provide invented constants.

5.4 Radio communication and ESP-NOW boundary

Espressif documents ESP-NOW as connectionless Wi-Fi communication using vendor-specific action frames. It provides peer send and receive interfaces; application-level routing remains the project's responsibility [7]. The ESP-IDF guide also documents callback-based send/receive processing and warns against long operations in the Wi-Fi task callbacks. A future implementation should enqueue received frame metadata for a lower-priority task rather than execute tree inference inside a time-critical callback.

Frame length limits and peer compatibility depend on the selected ESP-IDF/ESP32 version. The project does not choose a version or board, so the report does not specify payload maxima or radio throughput. Those details must be read from the exact target documentation and tested against the final frame size.

Radio success callbacks should be interpreted carefully. A link-layer transmit result may not equal application-level delivery at the gateway. The application needs a response tied to packet ID if its label is end-to-end or ACK success. The training target must match the measured event: the stored model target is simulated ACK success on a single link attempt.

5.5 Packet framing and interfaces

Field	Purpose	Design state
protocol_version	Schema selection and compatibility	1 byte proposed; encoding unselected
source_id	Origin identity	Width/encoding unselected
destination_id	Gateway or destination	Width/encoding unselected
packet_id	Duplicate detection and trial accounting	Counter width/wrap behavior unselected
source_timestamp	Source-side relative time	Clock basis and units unselected
hop_count	Observed forwarding count	Width and overflow handling unselected
hop_limit	Loop bound	Initial value to be measured/selected
payload_length	Frame bounds checking	Width and max-frame rule unselected
payload	Application data	Sensor format unknown
path_history	Visited-node check	Bounded encoding and size unknown
integrity_field	Corruption/authentication handling	Checksum/MIC choice unselected

Table 5.2: Proposed frame fields; layout remains to be implemented

A frame parser should treat all lengths as untrusted input. It should verify minimum header size, declared payload length, known protocol version, valid destination, and permitted hop limit before accessing fields. The parser should reject trailing or truncated data unless the encoding explicitly allows it. It should log a reason code without printing secrets or flooding the radio.

Logical addresses and radio peer addresses should be mapped through a configuration structure. That makes route decisions independent from hard-coded MAC strings and supports repeatable role assignment. The mapping needs a versioned record and should be captured with each trial.

5.6 Firmware architecture

A layered ESP32 firmware design separates platform initialization, radio transport, frame codec, neighbor table, measurement estimator, route policy, packet cache, gateway logger, and application workload. The Python repository has conceptual counterparts, and the INET project has a separate C++ simulation application; neither supplies ESP32 firmware modules. The proposed layers make ownership and test boundaries explicit.

Initialization should validate configuration before starting radio traffic. A node should set its logical ID, initialize storage and timers, configure radio, add known peers or begin discovery, register callbacks,

allocate bounded queues, and expose a local diagnostic state. A failure in one stage should prevent the node from advertising itself as ready.

Receive callbacks should copy only bounded frame data or enqueue a safe reference according to the SDK's lifetime rules. A worker task can parse, deduplicate, update metrics, request a route decision, and queue forwarding. Send completion callbacks should update attempt counters and wake the worker. This design must be checked against the specific ESP-IDF version and API contract.

The gateway logger should write append-only trial records or stream them to a host that saves raw data. Record packet IDs, receive time, path, hop count, link samples, route reason, and trial metadata. If a device restarts mid-run, retain a boot/session ID so counters and monotonic time can be interpreted correctly.

Firmware module	Responsibilities	Verification target
Board support	Clock, serial, watchdog, GPIO only if needed	Startup log, board identity
Radio adapter	Peer setup, send/receive callbacks	Two-node bidirectional test
Frame codec	Encode/decode bounded frame	Golden vectors and malformed-frame rejection
Neighbor manager	Discovery and freshness state	Expiry and rejoin behavior
Metric estimator	ACK/RSSI/window counters	Known scripted probe sequences
Route policy	Baseline and ML scoring	Python parity and decision logs
Forwarder	Hop/path/dedupe/queue handling	Loop, duplicate, limit tests
Gateway logger	Receive accounting and export	CSV field validation

Table 5.3: Proposed firmware module boundaries

5.7 Power design and energy budget method

The planned pack comprises four 18650 lithium-ion cells rated 3.7 V nominal and 2000 mAh each, arranged as a 4S string in two dual-cell holders whose actual terminal wiring and balance taps must be verified. Arithmetic gives 14.8 V nominal, 16.8 V at 4.2 V/cell full charge, 2000 mAh series capacity, and 29.6 Wh nominal energy. These are design calculations, not measured pack properties. Cells must be matched and inspected. The holder pair must expose the four series-cell junctions required by the selected BMS; combined pack leads alone are insufficient.

The current design omits a fuse. Proposed battery path is cell string → compatible 4S balancing/protection BMS → main switch → four separate LM2596 buck-converter inputs in parallel. Each module has its own adjusted and measured 5.0 V output feeding one ESP32 board; converter outputs are not tied together. Omitting a fuse removes an important layer of overcurrent protection. Verify cell matching, wire gauge, connector limits, BMS protection/current ratings, and fault behavior before energizing. The switch must have a suitable DC rating at the pack voltage and load.

Use a charger explicitly compatible with a four-series lithium-ion pack and its required 16.8 V CC/CV termination. A protection/balancing BMS is not a charger; a 1S TP4056 board is not suitable for charging the complete 4S pack. The candidate BMS listing advertises balancing and 40 A, but its stated charging-voltage range of 16.8–18.1 V is ambiguous and must be resolved against a trustworthy datasheet before use.

The pack energy estimate is $E = 4 \times 3.7\ V \times 2.0\ Ah = 29.6\ Wh$ nominal. It does not predict runtime: converter efficiency, board load, radio duty cycle, indicator/sensor current, cell condition, and cutoff behavior are not measured. At 5.0 V, each board's allowable load must be established from its measured workload and the selected module's thermal/input limits.

$$P(t) = V(t)I(t);\ \ E \approx \Sigma_k V_k I_k \Delta t_k \qquad (5.1)$$

Measurement field	Required record	Reason
Voltage	Probe location, value, sampling rate	Energy calculation basis
Current	Range, sensor, calibration, sample rate	Capture transient and average load
Trial interval	Start/stop event and duration	Comparable energy window
Delivered payload	Gateway byte count	Energy per delivered bit/packet if reported
Node role/state	Source/relay/gateway, radio duty state	Separate role-specific consumption
Supply method	USB/bench/battery and regulator	Interpret measurement boundary

Table 5.4: Power measurement worksheet

5.8 Assembly and integration plan

Phase A: inspect each board/module and identify component ratings; verify the four-cell holder wiring and BMS balance taps/pinout against selected documentation before loading cells. Phase B: with no ESP32 attached, validate BMS protection, compatible charger arrangement, DC switch rating, polarity, and each buck converter's 5.0 V output under representative load. The current design has no fuse, so review overcurrent protection and wiring limits before energizing. Phase C: connect one board at a time after rail stability and USB/external-power isolation are verified.

Phase F: integrate the Node 1 sensors, OLED, and each node's traffic-light indicator only after their voltage, logic levels, pin requirements, and driver needs are verified. Phase G: add raw logging, fixed-baseline forwarding, and controlled route failure/recovery. Phase H: collect measured observations and evaluate the model offline. Phase I: consider embedded inference only after feature parity and resource checks. Phase J: run paired physical trials and archive raw logs. Every phase remains planned until its records exist.

Each phase should have an entry condition and a recorded exit result. If packet framing is unreliable, do not interpret route performance. If ACK correlation is ambiguous, do not use those labels for model training. If the embedded model does not fit resource limits, preserve offline inference as a comparison and report the deployment constraint honestly.

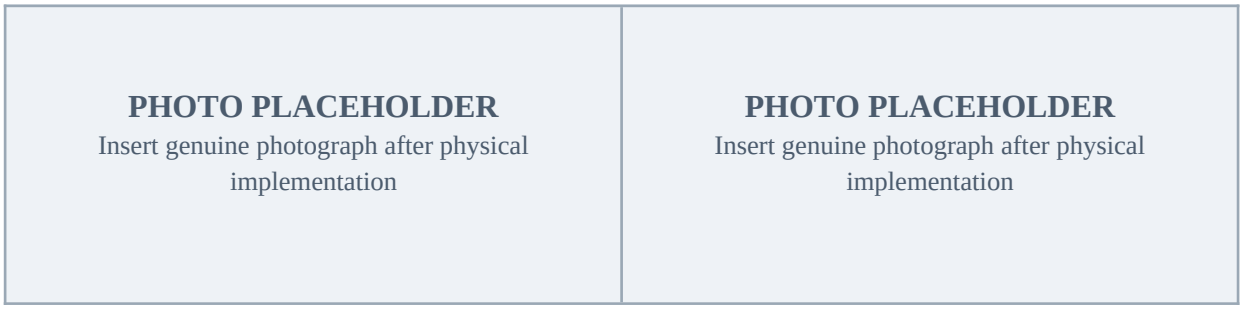


Figure 5.3: Physical assembly of the four-node IoT mesh prototype (photograph to be inserted after implementation).

Figure 5.4: Physical measurement and gateway logging arrangement (photograph to be inserted after implementation).

5.9 Hardware debug and acceptance approach

Debug evidence should progress from local serial logs to peer-level send/receive, gateway application receipt, relay path, and complete experiment output. A send API returning success is not sufficient proof that the application destination received a packet. Use IDs and logs on both ends to identify where a frame stopped.

Acceptance needs objective criteria defined before collecting final results. Examples include a minimum number of valid packets, no malformed frames under a fixed workload, correct hop accounting, and

deterministic route selection for an unchanged topology. Numeric thresholds should be selected by the project supervisor and based on requirements; this report does not invent them.

6 Routing Algorithm and Machine Learning Methodology

6.1 Baseline routing implementation

The Python baseline constructs a graph containing available nodes and edges, computes shortest paths from the current node to the gateway, and chooses the lexicographically minimum path among shortest candidates. The next vertex is returned with reason `minimum_hop`; if no route exists, it returns no neighbor and reason `route_unavailable`. This deterministic tie rule is relevant when two corridors have equal hop count.

The baseline is evaluated using the same graph and packet-level per-edge random draw key as the ML mode for a given packet and edge. This common-random-number design improves comparability when both methods traverse the same edge. If routes differ, they encounter edge-specific draws; the comparison remains paired by seed and workload but individual packet outcomes are not guaranteed to be coupled identically across paths.

6.2 Candidate eligibility and loop control

The ML engine sorts neighbors before evaluating them. It rejects neighbors present in packet history and edges marked unavailable. It computes a shortest-hop tail from the neighbor to the gateway, rejects missing tails and tails that return through the current node, and forms a feature row for the current link. Probability calculation errors of selected types cause that candidate to be skipped.

If all candidates are skipped or scoring produces no valid candidate, the engine calls minimum-hop routing and returns its next hop with a baseline-fallback reason. The simulator independently rejects a next hop already in packet history and enforces the packet hop limit. These safeguards are at different layers: candidate filtering, packet-level loop prevention, and hop budget. Hardware needs equivalent checks and tests; a Python helper does not implement a bounded persistent cache on a board.

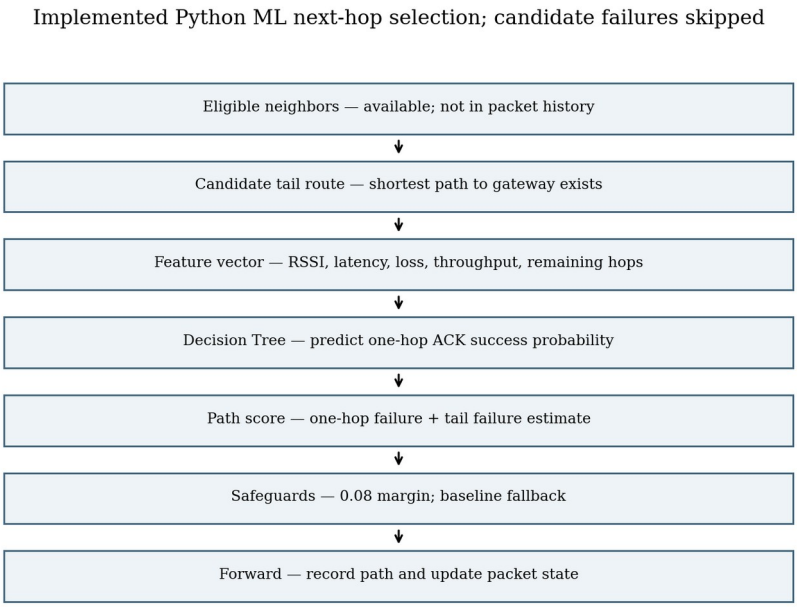


Figure 6.1: Implemented Python next-hop selection sequence reconstructed from routing/engines.py and simulator/core.py.

6.3 Link feature construction

Feature	Code meaning	Unit	Caveat
rssi_dbm	Synthetic/configured received signal strength	dBm	clipped in generator
latency_ms	Configured delay adjusted by modeled congestion	ms	not measured RTT
loss_estimate	Loss probability plus congestion adjustment	fraction 0–1	bounded
throughput_kbps	Capacity scaled by congestion	kbps	estimated from synthetic capacity
remaining_hops	Shortest-hop tail distance from candidate	hops	computed from graph

Table 6.1: Current model feature vector

The feature transformation uses capacity to compute an adjusted throughput value. It combines stored congestion with an experiment load term and clamps derived congestion. The transformations are defined in simulator/core.py and applied both during synthetic data generation and runtime scoring. This consistency is important for model inputs, but the values remain simulator constructs.

An embedded feature pipeline must reproduce units, bounds, order, and missing-data behavior. A byte-perfect schema should include a version identifier. If the firmware uses measured RTT rather than synthetic per-link latency, the model input distribution changes and the model requires retraining or a justified calibration process. Passing a physical value into a feature trained on a different meaning is not a valid deployment.

6.4 Synthetic observation generation

The generator defaults to 60 run IDs and 90 rows per run, producing 5,400 observations. It cycles through four named conditions: normal, degraded, busy, and strong. It samples graph edges, adds random perturbations to configured RSSI, loss, and latency, generates a congestion value, samples a remaining-hop distance, transforms features, and draws a Bernoulli success label from the adjusted synthetic loss/congestion expression.

The target `delivery_success` identifies a single simulated link attempt. It is not a gateway delivery label, end-to-end path label, or hardware ACK. The generator stores `run_id`, `condition`, and `synthetic=True`. This schema allows run-grouped splitting and makes synthetic provenance explicit.

The saved dataset distribution is class-imbalanced by condition. Strong-condition data contains 46 failure labels and 1,304 success labels in the 15 held-out run IDs. The stored model evaluation confusion matrix has zero true positives for the failure class, despite high overall accuracy. That is a direct warning about threshold utility and class sensitivity under the held-out condition.

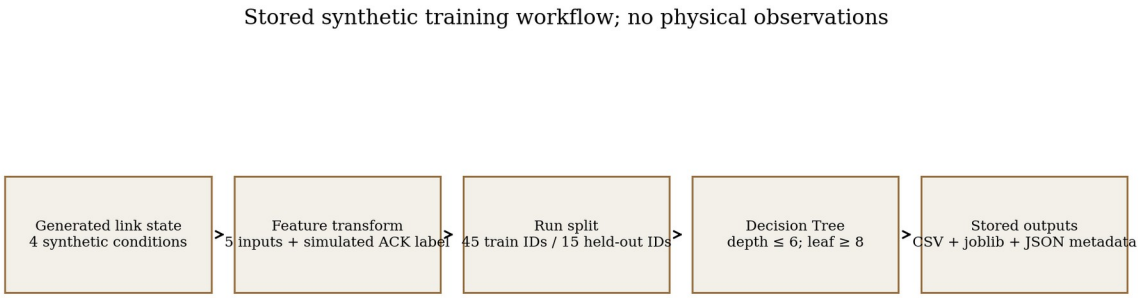


Figure 6.2: Synthetic training and evaluation workflow from ml/pipeline.py; all current rows are synthetic.

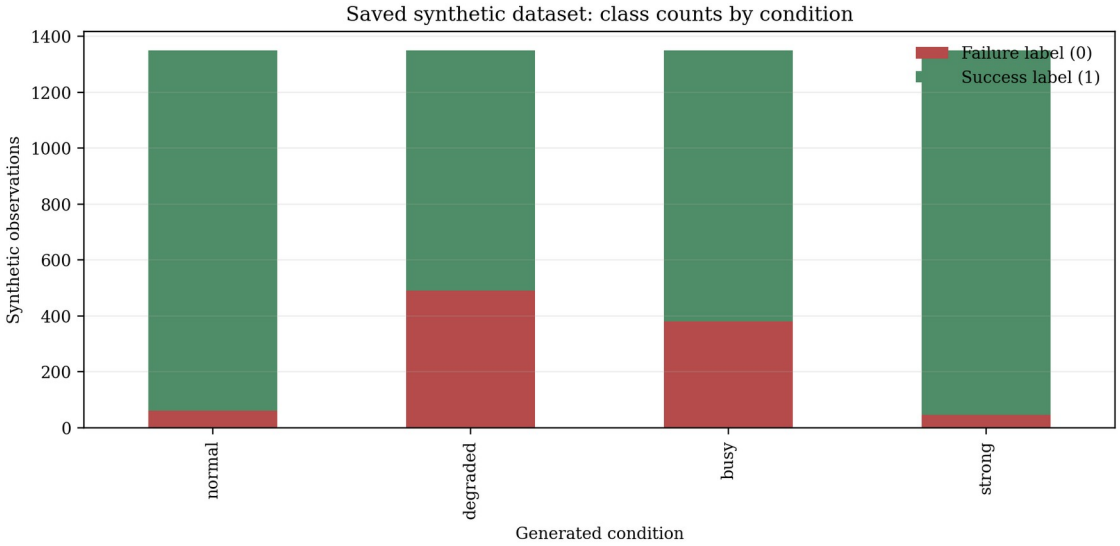


Figure 6.3: Saved dataset class counts by generated condition; bars are counts from data/link_observations.csv.

6.5 Dataset split and model training

The pipeline splits by complete run ID. With 60 runs, boundary is int(60×0.75)=45: run IDs 0–44 are training and 45–59 are test. The generator's condition schedule makes these training conditions busy, degraded, and normal; the held-out group is strong. This avoids random row leakage from the same run, but it confounds the held-out condition with a distribution shift: all test rows come from a condition not present in the training subset.

The fitted classifier uses maximum depth 6, minimum leaf size 8, balanced class weights, and a fixed random state. Class weighting changes training loss emphasis but cannot create unobserved examples or guarantee failure recall. Model evaluation includes accuracy, precision, recall, F1, class-specific failure

metrics, confusion matrix, ROC-AUC when both classes appear, and per-row inference time. The saved timing is an offline Python measurement, not embedded inference time.

The stored JSON says seed 3; training function default is 17 and the README train command uses the function default. The checked-in dataset's values should be matched against deterministic regeneration before assigning its seed. The raw experiment rows use seeds 17 through scenario/trial offsets. A model generated during `run_suite` could be retrained in memory with an experiment seed, but the saved JSON's seed 3 is not evidence that it is the model used for the stored outputs. This mismatch remains unresolved.

Split	Run IDs	Conditions represented	Rows
Train	0–44	busy, degraded, normal	4,050
Test	45–59	strong only	1,350
Total	0–59	four generated condition names	5,400

Table 6.2: Saved model metadata split; rows derived from run count and observations per run

6.6 Decision Tree model and target

A classification tree partitions feature space with threshold decisions and assigns class probabilities from terminal leaves. In this saved model, depth is capped at six and minimum samples per leaf at eight during fitting. The actual serialized tree contains 95 nodes and 48 leaves; these structural properties were inspected from the stored joblib artifact. A shallow tree can be represented as threshold tests, but the exact conversion must preserve feature ordering, comparison operators, missing-value behavior, and probability calculation.

The feature importance vector from the saved tree assigns most split importance to `loss_estimate` (approximately 0.776), followed by latency, RSSI, throughput, and remaining hops. This is an internal impurity-based attribute of the fitted tree; it does not establish causal importance, robustness, or reliability on physical measurements. It can help identify which feature distributions need review, but should not be presented as a radio-domain discovery.

The target is probabilistic one-hop success. Because the strong held-out condition has low failure prevalence, an always-success-like decision can yield high accuracy while missing every failure. Evaluation should emphasize failure recall, precision-recall trade-off, threshold selection, calibration, and cost of false negatives. The current saved model provides evidence that this evaluation concern is material.

Metric	Stored value	Interpretation limit
Accuracy	0.9607	Dominated by success class in held-out strong data
Success precision	0.9657	Positive predictive value for success label
Success recall	0.9946	Misses 7 of 1,304 actual successes
Failure precision	0.0000	No predicted failures, so no correct failure prediction
Failure recall	0.0000	Misses all 46 held-out failures
ROC-AUC	0.7350	Ranking metric; does not fix chosen class threshold
Confusion matrix	[[0,46],[7,1297]]	Rows actual [fail,success], columns predicted [fail,success]

Table 6.3: Stored synthetic classifier evaluation

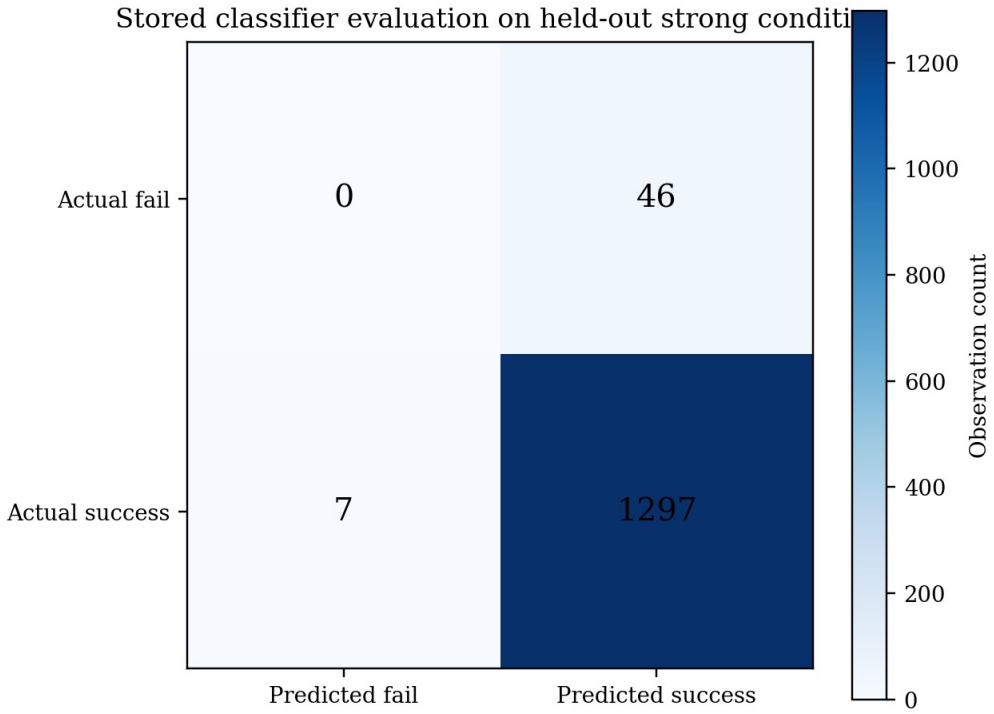


Figure 6.4: Confusion matrix from models/delivery_tree.json; held-out split contains only strong-condition runs.

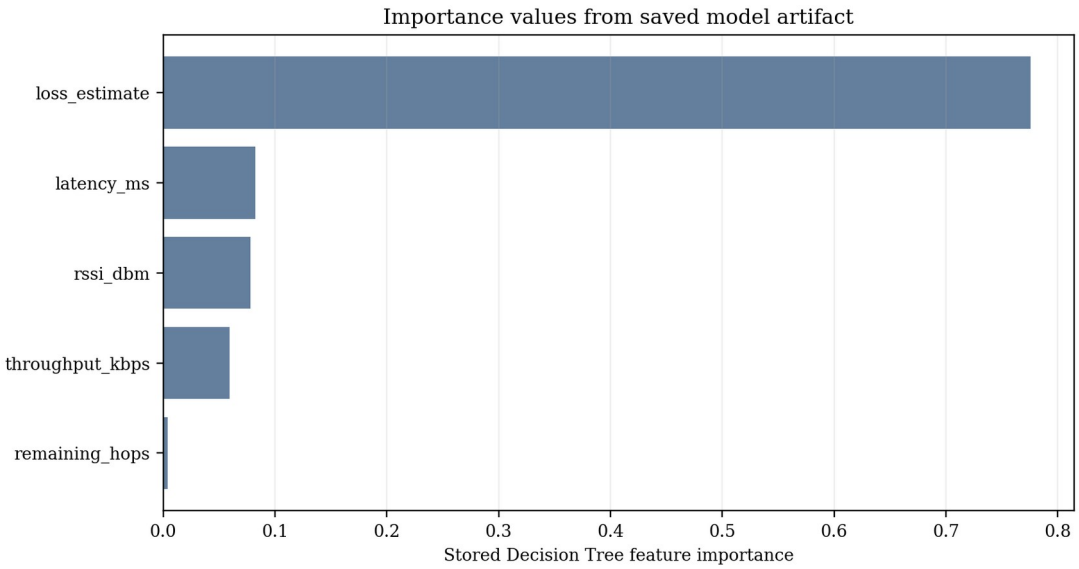


Figure 6.5: Feature importance values from saved DecisionTreeClassifier; values describe fitted tree splits only.

6.7 Inference and candidate cost

For each eligible neighbor, the scorer predicts the probability for class 1. It then estimates tail success by multiplying (1−loss probability) along the shortest-hop tail. Candidate score adds current-link failure risk and tail failure risk. This can prefer a path whose immediate edge is predicted to succeed and whose remaining configured links are comparatively reliable. The classifier probability and tail estimate have different origins: one is learned from synthetic examples; the other is calculated from configured graph loss probabilities.

$$P_tail(v) = \Pi_{(i,j)\in P_tail(v)} (1 - \ell_{ij}) \quad (6.1)$$

Here ℓ_{ij} is the simulator's configured loss probability for edge (i,j), not a measured physical loss estimate. The product assumes edge success probabilities combine independently. Correlated interference, shared channel contention, and common node failures violate that simplifying assumption. The simulator does not model those correlations.

6.8 Hysteresis and fallback

The previous next hop can be retained if it remains a valid scored candidate and its cost is no more than 0.08 above the current best. This is a static cost margin. It is not a dwell timer, temporal confirmation window, or full route-flap controller. The integration plan suggests adding dwell/confirmation behavior for hardware; that would be a new design feature and should be evaluated separately.

```
if no valid ML candidate:
    return minimum_hop_next_hop(...)  # baseline fallback

best = candidate with smallest score
if previous_candidate exists and previous.score <= best.score + 0.08:
    keep previous candidate
else:
    choose best
```

6.9 Complexity, deployment, and model risks

For a node with d neighbors, the route engine evaluates up to d candidates. Each candidate asks for a shortest path in the current graph and a model prediction. In a general graph, shortest-path complexity depends on the algorithm and representation; the prototype repeatedly creates an available-edge graph and enumerates shortest paths for deterministic tie resolution. This is acceptable for a small simulator but may be inefficient if directly scaled.

Tree traversal is bounded by model depth. The saved tree depth is six, so a single classification requires at most six threshold levels after features are prepared. This count does not include radio measurement, candidate table maintenance, data conversion, or model loading. No ESP32 timing, memory, flash, or power measurements exist.

Primary deployment risks are feature drift, stale values, class imbalance, false confidence under distribution shift, inconsistent feature semantics, and model/runtime mismatch. Safeguards should include feature validity checks, model versioning, baseline fallback, decision logging, and exact parity tests against Python vectors.

6.10 Evaluation metrics for classification

$$\text{Precision}_1 = \text{TP}/(\text{TP}+\text{FP});\ \text{Recall}_1 = \text{TP}/(\text{TP}+\text{FN});\ \text{F1}_1 = 2\cdot\text{Precision}_1\cdot\text{Recall}_1/(\text{Precision}_1+\text{Recall}_1) \tag{6.2}$$

For failure detection, treat class 0 as the positive class when computing failure precision and recall. Always state the class convention and confusion-matrix orientation. Accuracy is total correct predictions divided by sample count. If the test condition is imbalanced, accuracy may be dominated by common success observations. ROC-AUC assesses ranking across thresholds but does not select a safe operating threshold for routing.

7 Simulation Environment and Experimental Methodology

7.1 Python simulation objective and topology

The Python simulator's objective is to compare packet forwarding under a deterministic shortest-hop baseline and an ML-assisted next-hop scorer on a graph with alternative paths. The default six-node topology contains source S, gateway G, relay nodes R1–R4, two three-hop corridors, and two cross-links. Nodes can be added beyond six and attach to the relay core. Link attributes are copied into each scenario graph before packet processing.

The graph is not a geometric radio model. The simulator does not derive connectivity from distance, RSSI propagation, or antenna conditions. Its default edge attributes are set from function parameters and can be modified by scenarios. The network graph is an explicit abstraction for routing policy tests.

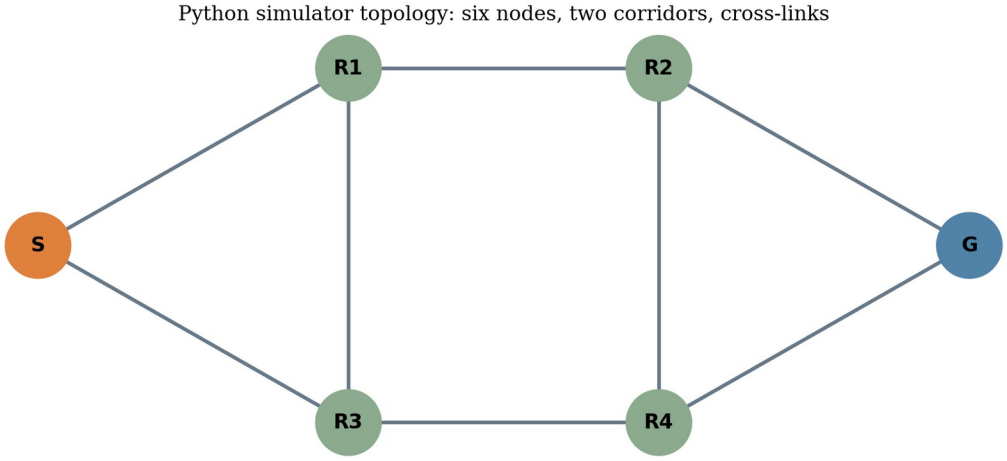


Figure 7.1: Python graph from simulator/core.py; link attributes are synthetic/configured.

7.2 Packet process and link model

Packets are generated at creation times pid/rate, where pid is packet index and rate is packets per second. A simulation clock advances to the next creation time, then accumulates link delay while forwarding. The packet stores source, destination, creation time, hop count, hop limit, path history, and payload size. The simulator records delivery status, latency for delivered packets, hop count, route, and payload bytes.

Each transmission obtains a seeded random draw based on global seed, packet ID, and canonical edge index. If the draw falls below the adjusted loss probability, the packet is marked lost. The same packet/edge key supports common random draws when both routing modes use the same link. Delay is configured latency adjusted by traffic load and link congestion. There is no queue scheduler or per-packet MAC contention model.

The Python throughput denominator is an offered-workload observation duration based on packet count and generation rate. Payload bits are calculated from delivered packets. Latency statistics use delivered packets only; undelivered packet latency is missing. Therefore a lower mean delivered latency can coincide with fewer delivered packets, and the delivery count should accompany latency summaries.

7.3 Experiment scenarios and pairing

Scenario	Code transformation	Interpretation
normal	No additional topology change	Reference synthetic operating point
degraded	Weakens S–R1 RSSI/loss/latency attributes	Tests alternate-quality response
preferred_link_failure	Marks S–R1 unavailable	Tests alternate route under failed edge
high_loss	Adds 0.30 loss probability up to a cap	Tests broad synthetic loss stress

Scenario	Code transformation	Interpretation
high_latency	Triples configured link latency	Tests timing sensitivity
congestion	Sets congestion feature and offered load	Synthetic load/metric adjustment
recovery	Disables S–R1 first half; restores at midpoint	State-toggle recovery abstraction

Table 7.1: Python scenario definitions

The runner loops through seven scenarios and five trials. Trial seed is base seed plus scenario index times 1,000 plus trial index. For each scenario/trial it runs `baseline` and `ml` with the same trial seed, so each pair shares scenario and seed. The raw file therefore has 70 algorithm-run rows and 35 paired scenario/trial units. It is inaccurate to describe 70 as independent paired experiments.

The aggregate file has one row for each scenario/mode combination and contains mean, sample standard deviation for selected metrics, and mean recovery time. Some metrics are near-constant within scenario because the simulator is deterministic apart from seeded delivery draws and configured transformations. Five trials are a small descriptive sample; no hypothesis test is encoded in the repository.

7.4 Stored output inventory and row interpretation

Artifact	Rows / records	Purpose	Caveat
trial_results.csv	70 algorithm-run rows	Scenario, trial, seed, mode, packet metrics	35 paired conditions; each mode is one row
aggregate_results.csv	14 group rows	Mean/SD by scenario and routing mode	Derived from stored raw CSV; provenance not embedded
link_observations.csv	5,400 observations	Synthetic training observations	All rows marked synthetic
delivery_tree.json	One metadata record	Stored model evaluation	Seed 3; failure recall 0
delivery_tree.joblib	Serialized tree	Stored classifier object	No embedded deployment artifact
INET trial_results.csv	25 paired rows	Five seeds × five conditions, AODV and Decision Tree	Curated summary; full raw run directories remain local
INET aggregate_results.json	Five condition groups	Means and sample SD across matched seeds	Simulator-specific; not physical measurements
MeshRoutingApp and generated header	C++ application and tree evaluator	Packet-level route choice and safe forwarding	INET simulation only; no ESP32 firmware

Table 7.2: Saved simulation and ML artifacts

In reviewing a raw record, `scenario`, `trial`, and `seed` define the pairing key; `routing` identifies the policy. PDR and packet-loss rate use packet counts. Latency and throughput are aggregated from delivered packets and observation duration respectively. `recovery_time_s` is NaN outside recovery or censored when no post-recovery delivery is recorded. The CSV has no project revision, run timestamp, dependency lock, or command string, so the exact origin of the saved files is not independently established.

7.5 OMNeT++/INET configuration

The parent INET project declares a six-node stationary topology with a gateway, source, and four relays, two candidate corridors, 2 Mbps configured bitrate, unit-disk reception, and a 230 m nominal communication range. Its Normal configuration uses AODV and UDP application traffic. The source sends 128-byte messages every 200 ms from 2 s to 58 s; the gateway collects them on a UDP sink. IPv4

TTL is configured as 8. A separate simulation/inet/ml-routing project adds MeshRoutingApp and generated C++ Decision Tree inference while retaining the parent AODV configuration for comparison.

The parent AODV scenario configurations include reduced range, competing traffic with interference, higher offered load, lower bitrate, a second UDP flow, relay crash, preferred relay crash, and crash/restart. In the separate ML routing project, the repository records completed local build and repeated simulation runs for healthy, degraded-reliability, increased-latency, relay-failure, and relay-recovery conditions, using five matched seeds (17–21). Curated summaries are checked in; full raw run directories remain local and are not included. These are simulator results, not physical measurements. Windows/Linux builds are unverified, AODV packet-level hop count is not measured, and routing-control overhead is not measured.

The parent project's `export_sca.py` utility parses scalar/statistic records and derives sent/received counts, PDR, loss ratio, mean delay when present, and receiver payload throughput. It labels hop count as unavailable from the stock UDP sink and directs route-change/recovery analysis to route logs. The ML routing project also records packet events and route decisions for its own experiments. Software tests were run for the ML routing project; runtime integration tests that require an activated OMNeT++/INET environment are separately identified as skipped where applicable.

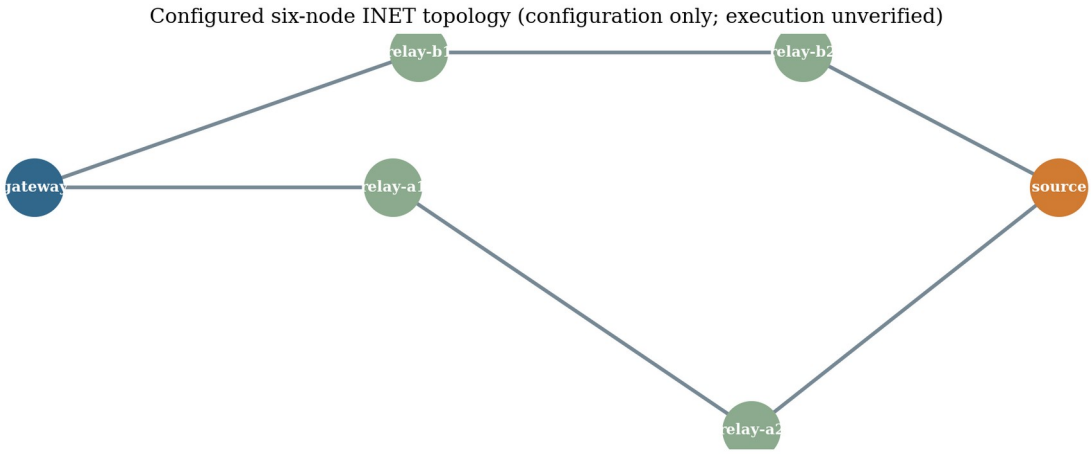


Figure 7.2: Six-node INET configuration layout; visual restatement of INI node coordinates, not a run screenshot.

7.6 Data collection and analysis procedure

For the Python phase, retain raw trial rows and aggregate rows together. Derive means and sample standard deviations by scenario and mode. Verify that raw baseline and ML rows match on trial and seed before computing paired differences. Do not average across scenarios because their transformations represent different conditions.

For INET, preserve raw scalar, vector, routing-table, and event output. Verify source sent count and gateway receive count against raw `.sca` data before relying on exported PDR. Use packet sequence vectors and route logs to reconstruct hops and route changes only when definitions are explicit. The exporter does not produce every metric requested by the hardware plan.

For future hardware, retain a trial manifest and raw gateway log. Define packet schedule, payload, topology, distance, radio configuration, firmware version, test order, environmental notes, and any controlled attenuation. Prefer paired trials that use the same configuration and alter only routing mode. Repeat enough times to estimate variation; determine sample size based on practical constraints rather than implying five trials are universally adequate.

7.7 Reproducibility protocol

```
python train_model.py
python run_experiments.py --trials 5 --packets 100 --seed 17 --nodes 6
pytest -q  # Python project test command
```

The Python commands describe the earlier simulator workflow; the INET ML project README documents the model-generation, build, AODV, Decision Tree, and paired-trial commands used for the current curated results. A reproducible run should write to a new directory, save exact output files, capture Python and OMNeT++/INET versions, record the source revision or archive checksum, and retain the command line. The older Python dataset/model seed mismatch remains unresolved and must not be conflated with the separately seeded INET route-choice model.

7.8 Simulation boundaries

- Python sequential processing abstracts radio contention, queue scheduling, collision, MAC retries, mobility, calibrated propagation, interference, and battery behavior.
- The Python RSSI/capacity/loss values are synthetic or configured; they are not measurements.
- INET's range-based unit-disk model is not calibrated ESP-NOW behavior and does not include the project ML scorer.
- Curated repeated-run summaries are present for the separate INET ML project; the full raw run directories are not archived in the repository. The legacy AODV scalar-export workflow remains distinct from those curated ML/AODV packet-level comparisons.
- Stored Python CSV provenance does not identify an exact source revision or invocation manifest.

8 Results and Analysis

8.1 Evidence inventory

This chapter analyzes the checked-in Python aggregate and raw CSVs and saved model metadata. The 70 raw rows form 35 paired scenario/trial units; each unit has one baseline row and one ML row. Seven scenarios have five trial seeds each. The aggregate CSV contains 14 rows. These are recorded earlier Python simulator outputs; they are not physical tests and are not attributed to the unexecuted INET configuration.

The raw and aggregate values agree when grouped from the checked-in trial file. The dataset has 5,400 rows, 60 run IDs, and all records marked synthetic. The model metadata reports 45 training runs, 15 held-out runs, and a strong-condition test split. No hardware log or power trace is available.

8.2 Packet delivery results

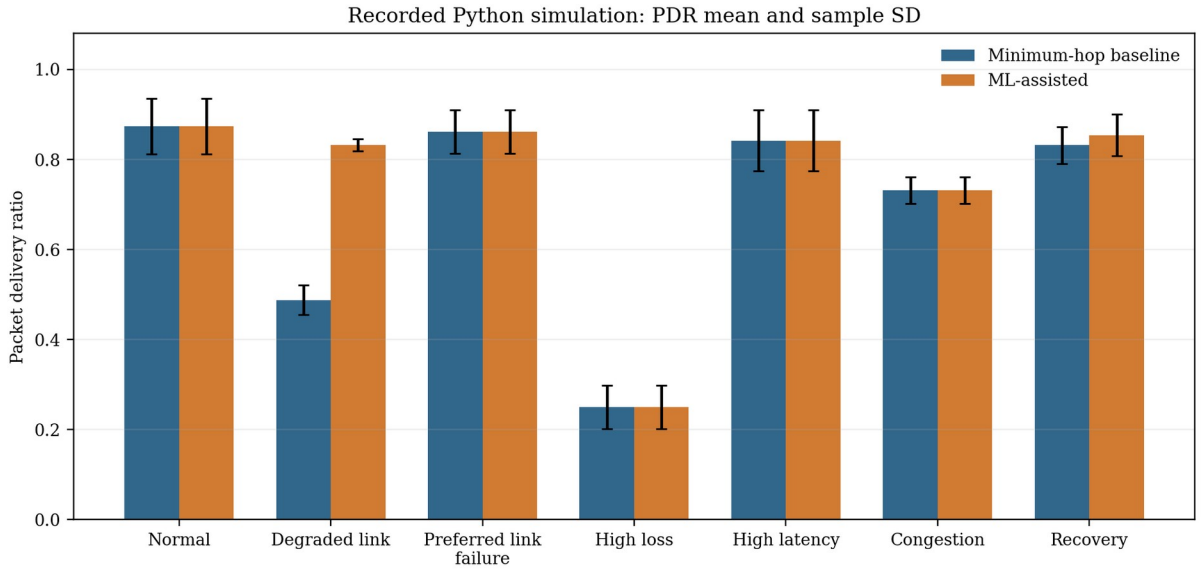


Figure 8.1: Stored Python simulation PDR means with sample standard deviations; five trials per mode and scenario.

Scenario	Minimum-hop PDR mean ± SD	ML PDR mean ± SD	ML − baseline mean
normal	0.874 ± 0.062	0.874 ± 0.062	+0.000
degraded	0.488 ± 0.033	0.832 ± 0.013	+0.344
preferred link failure	0.862 ± 0.049	0.862 ± 0.049	+0.000
high loss	0.250 ± 0.048	0.250 ± 0.048	+0.000
high latency	0.842 ± 0.068	0.842 ± 0.068	+0.000
congestion	0.732 ± 0.029	0.732 ± 0.029	+0.000
recovery	0.832 ± 0.041	0.854 ± 0.046	+0.022

Table 8.1: Stored Python paired aggregate PDR results

In five of seven scenarios, the displayed aggregate PDR means are equal between policies: normal (0.874), preferred-link failure (0.862), high loss (0.250), high latency (0.842), and congestion (0.732). In degraded-link, the stored ML mean is 0.832 compared with 0.488 for minimum-hop. In recovery, the stored ML mean is 0.854 compared with 0.832. These are scenario-specific observations from the saved synthetic runs, not a general performance guarantee.

The degraded scenario modifies only the preferred S–R1 edge's simulated attributes while leaving an alternate corridor available. The ML score can therefore select a different route based on synthetic

features; this may explain the stored difference, but packet-level route records should be inspected to establish exact causal paths. The aggregate alone does not show each packet's chosen route or whether a result depends on a small number of random draws.

High-loss PDR is low for both modes. The classifier's held-out evaluation is conducted on strong-condition training data and does not validate high-loss behavior. The high-latency scenario increases configured delay and preserves identical PDR means across modes. No energy or lifetime metric appears in the outputs.

8.3 Latency results

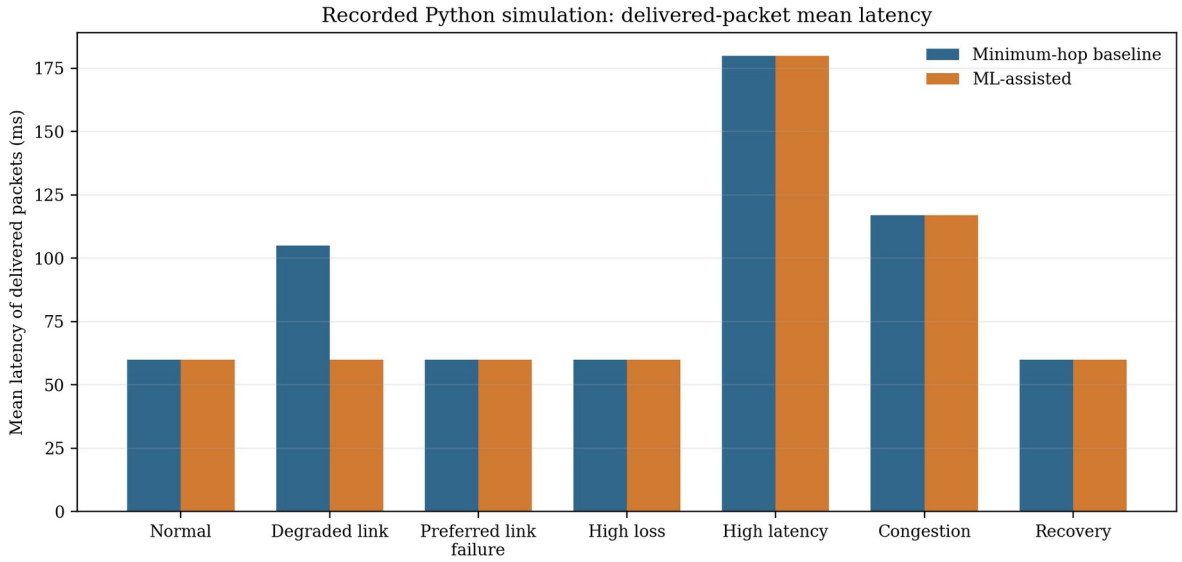


Figure 8.2: Mean latency for delivered packets in stored Python trials; failed packets are excluded from latency means.

Normal, preferred-link-failure, high-loss, and recovery means are 60 ms for both modes. Congestion reports 117 ms for both; high latency reports 180 ms for both. Degraded-link reports 105 ms for baseline and 60 ms for ML. The code's latency is accumulated configured per-link delay with scenario and load adjustment. It is not measured wall-clock latency.

Mean latency excludes undelivered packets. In scenarios with different PDR, comparison is conditioned on delivery and may describe different subsets of generated packets. Report delivered count with latency, and include percentiles only with adequate sample size and a clear calculation method. Existing aggregate output includes means and p95 summaries but does not provide confidence intervals.

8.4 Throughput results

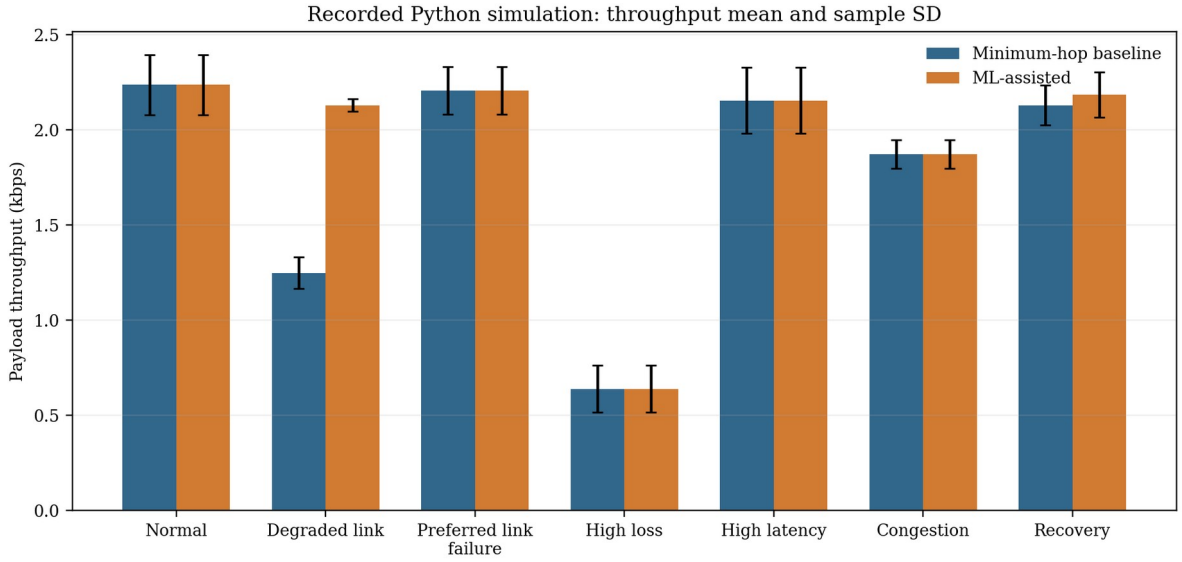


Figure 8.3: Payload throughput means and sample standard deviations from stored Python aggregate output.

Throughput follows delivered payload bytes divided by the simulator's offered-workload observation duration. In the degraded condition, ML throughput mean is 2.130 kbps versus 1.249 kbps for baseline. In recovery, means are 2.186 and 2.130 kbps. Other scenarios show equal displayed means. These values are software-derived payload rates using a 64-byte simulator packet default; they are not radio capacity or measured ESP32 throughput.

8.5 Route changes and recovery

The route-switch count in aggregate output is zero across modes/scenarios except recovery baseline, where mean count is 1.0; recovery ML is 0.0. The implementation tracks next-hop changes in a per-node current-route dictionary across successive packets. The count is not a direct measure of route convergence time, control overhead, or physical route flapping.

Recovery time is present only in the recovery scenario. Baseline mean is 10.04 s with sample SD 0.089 s; ML mean is 10.00 s with SD 0.000 s in stored aggregates. This time is from the simulated midpoint recovery to the first delivered packet whose ID is in the post-recovery half, measured using packet creation times. It is not the time from a detected physical link failure to restored radio delivery.

8.6 Stored model evaluation

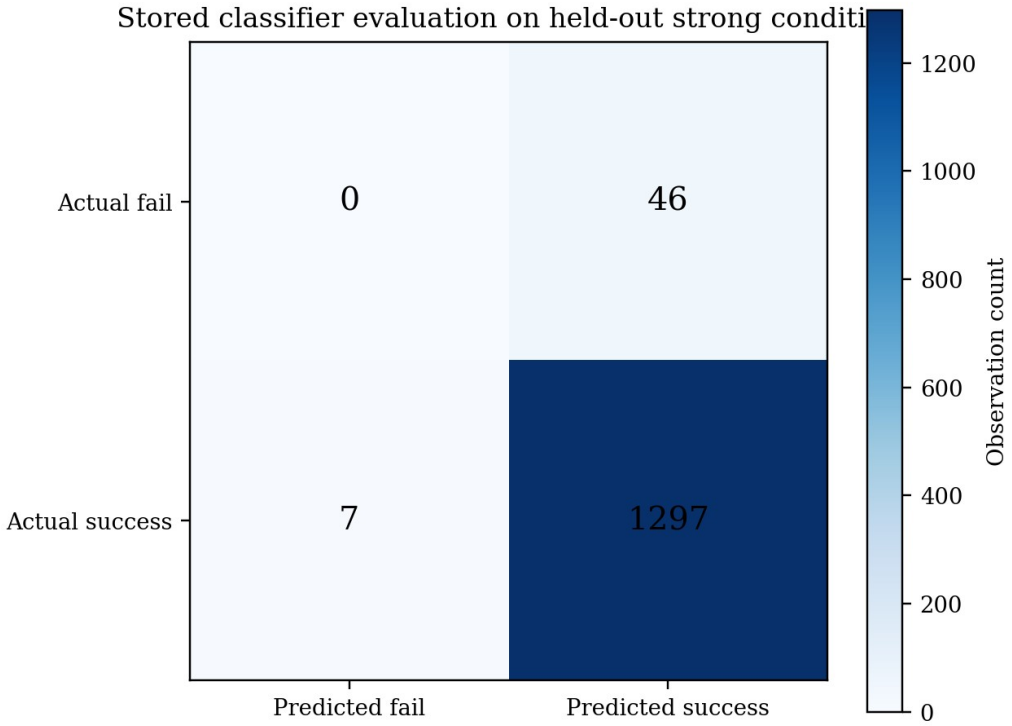


Figure 8.4: Stored classifier confusion matrix on held-out strong synthetic condition.

The model JSON records accuracy 0.9607, success precision 0.9657, success recall 0.9946, success F1 0.9800, ROC-AUC 0.7350, and failure precision/recall/F1 all 0. The confusion matrix is [[0,46], [7,1297]] using rows actual [failure,success] and columns predicted [failure,success]. The classifier therefore predicts zero failures in the held-out split and misses all 46 actual failures. Accuracy alone substantially overstates suitability for a policy that needs to identify weak links.

The held-out test contains 1,350 strong-condition observations, of which 46 are failures. A model could obtain high accuracy by favoring success in this class distribution. The stored result is not a physical validation and should motivate class-sensitive dataset collection, threshold analysis, calibration, and explicit failure detection criteria. It should not be hidden in a headline model score.

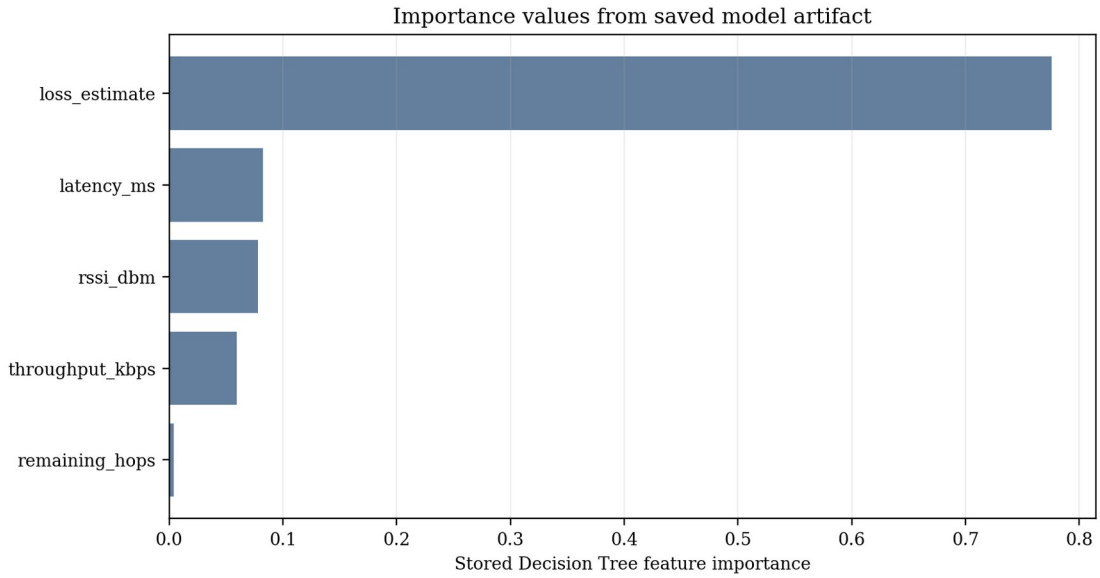


Figure 8.5: Stored model feature importances; values are tree split-importance statistics, not causal effects.

8.7 Interpretation by scenario

- Normal: both methods have the same stored mean PDR and latency, suggesting no observed benefit in this default synthetic condition. It does not prove statistical equivalence.
- Degraded: the stored ML aggregate is higher in PDR and throughput and lower in delivered-packet latency. Reproduction and path-level trace inspection are needed before interpreting this as policy effect.
- Preferred link failure: means match. The simulator makes the edge unavailable before forwarding; this is a deterministic topology change rather than packet-level link detection.
- High loss: both methods have low PDR; the held-out classifier has zero failure recall, so this scenario is not supported by the held-out model evaluation.
- High latency: both means match and latency rises to the configured synthetic level. It does not measure real network queueing.
- Congestion: both means match under configured load/congestion adjustments. No MAC scheduler or actual channel contention is represented.
- Recovery: small mean differences and different route-switch counts arise from the simulator's midpoint state restoration; interpret only as code-level behavior.

8.8 INET Decision Tree simulation results

The separate simulation/inet/ml-routing project runs a C++ MeshRoutingApp with generated Decision Tree inference and compares it with the AODV baseline. The checked-in repeated-trial summary contains five matched seeds (17–21) for each of five conditions. Values below are means with sample standard deviations across seeds. PDR is simulated gateway delivery divided by generated packets; delay is the recorded simulation delivery delay. These are packet-level OMNeT++/INET outputs under the configured topology and radio model, not ESP32 measurements.

Table 8.2: Five-seed OMNeT++/INET AODV and Decision Tree summary; values are simulation outputs.

Condition	AODV PDR (mean ± SD)	Decision Tree PDR (mean ± SD)	Mean delay AODV → Tree (ms)
Healthy	0.9993 ± 0.0016	0.9600 ± 0.0894	5.21 → 89.37
Degraded reliability	0.2900 ± 0.1615	0.6929 ± 0.0621	4293.50 → 4798.18
Increased latency	0.9986 ± 0.0032	0.9671 ± 0.0442	22.82 → 283.21

Condition	AODV PDR (mean ± SD)	Decision Tree PDR (mean ± SD)	Mean delay AODV → Tree (ms)
Relay failure	0.9986 ± 0.0020	0.9800 ± 0.0447	5.55 → 240.91
Relay failure + restart	0.9986 ± 0.0020	0.9800 ± 0.0447	5.55 → 240.91

Under degraded reliability, Decision Tree PDR averaged 0.6929 compared with 0.2900 for AODV, while mean delay was higher (4798.18 ms versus 4293.50 ms). In healthy, increased-latency, and relay-failure conditions, AODV had higher mean PDR and lower mean delay. The Decision Tree selected a three-hop delivered route in all reported trials; AODV packet-level hop count was not instrumented, so hop count is not a valid direct comparison. Throughput also favored the Decision Tree in degraded reliability (3547.4 versus 1484.8 bit/s) and favored AODV in the other conditions. This mixed result does not support a general superiority claim.

During relay restart, the Decision Tree application required an end-to-end probe reply before restoring the relay to its eligible set. The probe did not return in these trials, so the restarted relay remained ineligible and traffic continued on the alternate route. The reported 2.954 ± 4.031 s interval is time to first gateway delivery after failure, not proof of restored relay service. Active-probe bytes are excluded from overhead metrics, and routing-control overhead is not measured.

8.9 Missing performance evidence

Metric	Available evidence	Missing evidence
PDR / packet loss	Synthetic Python CSVs	Physical packets, radio ACK correlation, controlled repeat trials
Latency	Synthetic accumulated delay	Hardware timestamps/RTT method and clock basis
Throughput	Payload calculation from simulated packet count	Measured application goodput and airtime
Route changes	Python next-hop transition count	Firmware decision logs and convergence timing
Energy	None	Current/voltage instrumentation and defined interval
Network lifetime	None	Battery profile, duty cycle, sustained workload duration
Control overhead	Not modeled in Python metrics	Beacon, probe, routing and retransmission byte counts
INET packet-level results	Curated five-seed ML/AODV simulation summaries	Full raw run directories and validated physical measurements

Table 8.3: Result availability and missing evidence

8.10 Results chapter summary

Stored Python and INET outputs show scenario-dependent behavior. The earlier Python link-success classifier has zero failure recall on its held-out strong-condition split; the separate route-choice classifier has 96.67% accuracy on its synthetic held-out dataset. In five-seed INET comparisons, the Decision Tree improved PDR only in the degraded-reliability condition and had worse mean PDR in the other reported conditions. Together, these results support continued evaluation, not a claim that ML routing is generally better, energy efficient, or ready for embedded deployment.

9 Discussion, Limitations, and Hardware Validation

9.1 Interpretation against objectives

The objective to implement a software comparison is supported by source files and stored experiment outputs. The code includes a minimum-hop baseline and an ML scorer with eligibility and fallback checks. The objective to evaluate the model is partially supported by saved metrics, but the zero failure recall makes it unsuitable as a reliable failure detector in the stored held-out condition.

The objective to complete physical multi-hop validation is not met in the available repository. No hardware assembly, board identification, ESP32 firmware, raw radio observations, or prototype photographs were found; the included component images are supplier references. The INET AODV baseline configuration and separate C++ Decision Tree application have been built and run in the documented local environment, with curated five-seed summaries present. Those simulator results do not satisfy the physical validation objective.

The hardware plan should prioritize repeatable forwarding and measurement before model deployment. A baseline that reliably sends and logs packets provides the control condition needed to evaluate whether learned routing adds value. Introducing ML before reliable packet accounting would make later results difficult to attribute.

9.2 Dataset and model limitations

Synthetic feature values are sampled from selected distributions and transformed by the same code used for simulation. Their labels are generated by a Bernoulli success model. The learned tree can therefore reflect the synthetic label function rather than patterns in a physical radio. A model can perform well on synthetic held-out data while failing on real observations.

The test split holds out the strong condition entirely. This is useful for probing distribution shift but creates strong class imbalance and exposes failure recall of zero. Multiple evaluation regimes are needed: run-held-out within known conditions, condition-held-out tests, and ultimately physical held-out runs. Results should report class counts and thresholds.

The artifact seed mismatch complicates reproducibility. The saved model JSON seed 3 does not match README’s documented training default or stored experiment seed sequence beginning at 17. The dataset/model pair and the model used for trial generation must be traced. Until that happens, model metrics and routing output should remain attributed to their separate files.

9.3 Simulation limitations

The Python simulator is sequential and does not model channel scheduling, collisions, MAC acknowledgements/retries, interference fields, node mobility, or battery state. Its per-link Bernoulli loss is independent at each sampled attempt. It does not account for airtime consumed by discovery, probes, route advertisements, or model updates.

The INET design includes a more detailed network stack and AODV configuration, but run validity is unknown. Its unit-disk propagation, range, bit rate, offered load, and traffic definitions are simulation parameters. Even after execution, the result would remain model-based and would not directly validate the custom ESP-NOW implementation.

The common random draw design supports paired comparisons but not causal claims by itself. A statistical analysis should inspect paired differences per seed and scenario, not only unpaired mean values. Five pairs are not enough to characterize all random variability or support strong generalization.

9.4 Embedded and radio constraints

An embedded implementation must fit frame fields, neighbor table, duplicate cache, route state, model evaluator, and logging into the selected board's resource limits. None of those limits can be quantified until the board and firmware version are selected. The stored Python model's depth is small, but its conversion and runtime integration still need validation.

Radio callback scheduling, send pacing, peer count, frame length, security settings, and channel usage depend on the chosen ESP-IDF/ESP32 version [7]. The project's architecture should keep callback work bounded and route inference in a separate task. This is a proposed design response to documentation guidance, not an implemented feature.

Power measurements must distinguish node roles and radio states. A relay may transmit and receive more often than a leaf source. Model inference may have negligible or material cost depending on runtime and frequency; no assumption is justified without measurement. Current draw should be sampled during baseline and ML workloads, with the same power boundary and delivered workload.

9.5 Physical validation protocol

First, inventory boards and create a node-ID/MAC mapping. Record hardware and firmware revisions. Bring up radio peers without routing, then verify a fixed one-hop transmission using matched send and receive IDs. Next add a fixed multi-hop route and confirm that gateway logs contain each packet ID, path, hop count, and receive time.

After reliable forwarding, implement a baseline and establish a repeatable traffic schedule. Check packet generator count against gateway receipt count and drop logs. Add neighbor discovery and ACK/probe metrics; calculate loss and RTT over a predeclared rolling window. Save raw samples, missing-value states, and the window parameters.

Collect training data over separate trials and link conditions. Label a sample only when its probe/data attempt and ACK outcome can be paired. Separate training and testing by complete runs or conditions. Before ML routing, assess failure recall, calibration, confusion matrix, and threshold behavior on held-out data. If failure detection is inadequate, do not enable the model to override a safe baseline.

For paired performance evaluation, use identical topology, packet schedule, payload size, and physical placement across baseline and ML modes. Alternate order to reduce temporal bias. Test normal, degraded link, preferred relay outage, and recovery cases. Log every chosen next hop and reason. Report PDR, delivered latency distribution, throughput, hop count, route switches, recovery time, and control overhead. Add energy only with calibrated current/voltage instrumentation.

Test	Procedure	Recorded measures	Acceptance evidence
Board bring-up	Flash identity firmware; record serial and version	Board ID, boot count, radio configuration	All nodes stable and uniquely mapped
Peer connectivity	Bidirectional direct packets	TX/RX IDs, callback status, RSSI if exposed	Matched receive records across repeated attempts
Fixed-route forwarding	Source → relay → gateway	Packet ID, path, hop count, receipt time	No unexplained duplicate delivery; accounting reconciles
Loop/duplicate defense	Inject repeated ID and loop path	Drop reason, cache age, hop fields	Repeated/malformed cases rejected as designed
Link degradation	Controlled distance/attenuation change	RSSI, ACK/loss window, route choice	Raw condition and route response logged
Relay outage/recovery	Disable then restore selected relay	Outage marker, first alternate/returned path	Detection and recovery measured separately
Baseline vs ML	Paired traffic trials	PDR, latency, rate, switches, score/reason	Same trial config; uncertainty reported
Energy (optional)	Instrument supply over matched workload	V/I samples, duration, delivered payload	Measurement boundary and calibration recorded

Table 9.1: Planned hardware validation cases; none is claimed complete

9.6 Acceptance criteria and reporting

Acceptance criteria should be approved before final trials. At minimum, packet accounting must reconcile generated IDs with received, lost, rejected, or pending status; every forwarding decision must log its selected neighbor and reason; and the baseline must reproduce under a fixed configuration. A route result without raw packet identity is not sufficient for a delivery claim.

ML evaluation should include per-class precision, recall, F1, confusion matrix, ROC-AUC or precision-recall analysis where appropriate, calibration, and inference timing on target. Routing evaluation should report means and variation across paired trials. Energy claims require raw current/voltage data and a precisely described measurement boundary.

If physical tests fail or show no gain, report the conditions and failure modes. Negative or mixed results are valid engineering outcomes. They may indicate that simple minimum-hop is adequate in the tested environment, the model lacks representative features, model overhead is too high, or the topology does not provide a useful decision opportunity.

9.7 Risk controls

Risk	Impact	Control / evidence
Stale neighbor metrics	Wrong candidate ranking	Age limit, validity flags, record sample window
Model misses link failures	Continued use of weak edge	Failure recall gate; baseline fallback; route logs
Route loop or duplicate storm	Wasted airtime / delivery failure	History/hop limit/cache tests in firmware
Callback blocking	Missed radio events	Queue work to lower-priority task; measure latency
Radio condition drift	Unfair mode comparison	Pair and alternate trials; log placement/channel
Energy claim without instrument	Unsupported conclusion	Omit claim until calibrated measurement
Data/model provenance mismatch	Irreproducible score	Versioned manifest, hashes, matched artifacts

Table 9.2: Project risk controls

10 Conclusion and Future Work

10.1 Work completed

The repository contains a Python graph simulation, deterministic minimum-hop routing, a Decision Tree candidate scorer, synthetic link-observation generation, paired scenario runner, metrics, dashboard, stored model/data artifacts, and automated tests. Its saved CSVs provide 35 paired scenario/trial conditions across seven scenarios. A separate OMNeT++/INET project contains the C++ MeshRoutingApp, generated Decision Tree inference, route-safety checks, and curated five-seed comparisons against AODV across five conditions.

The parent repository contains a six-node INET AODV model configuration, scenarios, and a scalar exporter. The separate simulation/inet/ml-routing project has recorded local builds and completed repeated simulations for its Decision Tree application and AODV comparison. Its curated five-seed summaries are in the repository, while full raw run directories remain local. A hardware synopsis and integration plan describe ESP32/ESP-NOW as a planned testbed; ESP32 firmware and physical experiments are absent.

10.2 Conclusions supported by evidence

- Stored synthetic Python results show higher ML PDR in the degraded scenario and a small difference in recovery, with equal displayed PDR in the other five scenarios. Separately, five-seed INET results show higher Decision Tree PDR in degraded reliability but lower mean PDR in healthy, increased-latency, and relay-failure conditions. These are mixed, scenario-level simulation findings, not a general improvement claim.
- The saved model exhibits zero failure-class recall on its held-out strong-condition data. Its high accuracy does not establish reliable failure detection. The seed mismatch between stored metadata and documented/example trial provenance remains to be reconciled.
- No current evidence supports claims of physical ESP32 performance, on-device ML deployment, measured energy savings, network lifetime improvement, or final hardware validation. INET build and simulation execution are recorded for the ML routing project, but those results do not establish physical-network behavior.

10.3 Remaining work

- Resolve student/institution/guide information and confirm final hardware components.
- Reproduce model and Python trials with versioned source and run manifest; reconcile seeds.
- Reproduce and archive INET paired scenarios with raw outputs, run manifests, and environment versions.
- Implement stable fixed-route ESP32 forwarding and gateway logs.
- Collect real, labeled link observations; review class-sensitive model performance.
- Port inference only after parity and resource checks.
- Run paired baseline/ML hardware experiments and optional energy tests.
- Complete physical build and validation, then add actual prototype photographs, measured plots, and supervisor-approved hardware results while retaining the existing architecture diagrams and supplier-reference images.

10.4 Future extensions

After physical validation, future work could investigate richer link estimators, temporal smoothing, an explicit route-cost advertisement protocol, adaptive hysteresis, multi-objective energy/latency/reliability trade-offs, and larger or mobile topologies. These are possible extensions, not current implementation features. The first priority remains establishing evidence on the proposed small testbed.

A Repository and Artifact Inventory

The inventory below identifies the principal project files inspected for this report. It is intended to support review and reproduction; it is not a claim that every artifact was generated by the same revision.

Area	Principal files	Report use
Project definition	README.md; synopsis DOCX	Status, goals, planned ESP32 design
Architecture	ARCHITECTURE.md	Simplifications and module boundaries
Python simulator	simulator/core.py	Graph, packet lifecycle, scenarios
Routing	routing/engines.py	Baseline and ML scorer
Learning	ml/pipeline.py; models/*; data/*	Synthetic data and model metadata
Metrics	metrics/core.py	PDR, latency, throughput calculations
Experiments	experiments/runner.py; outputs/*	Paired run records and aggregates
Dashboard	app.py	UI and exposed controls
Tests	tests/test_system.py	Existing test definitions
INET	omnetpp-inet/src; simulations; tools; tests	AODV config and SCA exporter
Hardware plan	docs/ESP32_INTEGRATION.md	Proposed packet, metrics, safeguards
Reference material	papers/*.pdf; reports/MiniProject_RCC_v1 (1).pdf	Literature and report-format model
INET ML routing	simulation/inet/ml-routing	MeshRoutingApp and generated tree; five-seed results recorded; raw run folders local; no physical validation

Table A.1: Repository module map

The reports directory also contains a presentation and review checklist. Those files were inventoried but were not used as authority for report claims beyond their source-gap summary. The original demo PDF remains preserved.

This report retains the original 10-chapter structure, detailed analysis, tables, figures, and appendices. This refinement updates evidence statements and adds current INET implementation and repeated-run results while preserving the earlier Python findings as a separate evidence track.

B Data Dictionary and Output Schemas

B.1 Synthetic training dataset

Column	Type / unit	Meaning	Provenance
rssi_dbm	float / dBm	Generated received-strength feature	Synthetic
latency_ms	float / ms	Transformed per-link latency feature	Synthetic
loss_estimate	float / fraction	Transformed loss feature	Synthetic
throughput_kbps	float / kbps	Capacity adjusted by congestion	Synthetic
remaining_hops	float / hops	Candidate tail distance	Synthetic graph
delivery_success	int / 0 or 1	One simulated attempt outcome label	Synthetic Bernoulli
run_id	int	Generated run group for split	Generator
condition	string	normal/degraded/busy/strong	Generator
synthetic	bool	Marks data provenance	True for all stored rows

Table B.1: Training dataset schema

Training rows represent generated feature/label observations and must not be merged with future measured rows without a source column and compatible schema. Physical rows should add node/link IDs, firmware/board version, trial, timestamp, window length, ACK timeout, attempted packet count, missing-value flags, and environment metadata. Raw features should be preserved before normalized/model-input transformations.

B.2 Raw trial output

Column	Meaning	Interpretation
scenario	Scenario-run field	One row per algorithm execution
trial	Scenario-run field	One row per algorithm execution
seed	Scenario-run field	One row per algorithm execution
routing	Scenario-run field	One row per algorithm execution
pdr	Scenario-run field	One row per algorithm execution
packet_loss_rate	Scenario-run field	One row per algorithm execution
mean_latency_ms	Scenario-run field	One row per algorithm execution
median_latency_ms	Scenario-run field	One row per algorithm execution
p95_latency_ms	Scenario-run field	One row per algorithm execution
throughput_kbps	Scenario-run field	One row per algorithm execution
packets_generated	Scenario-run field	One row per algorithm execution
packets_delivered	Scenario-run field	One row per algorithm execution
route_switch_count	Scenario-run field	One row per algorithm execution
observation_duration_s	Scenario-run field	One row per algorithm execution
recovery_time_s	Scenario-run field	One row per algorithm execution

Table B.2: Stored trial-results schema

`scenario`, `trial`, and `seed` identify paired run conditions; `routing` separates baseline and ML. `pdr` and `packet_loss_rate` summarize packet outcomes. `mean_latency_ms`, `median_latency_ms`, and `p95_latency_ms` use delivered packets only. `throughput_kbps` uses delivered payload bits and modeled observation duration. `route_switch_count` counts changes in a forwarding node’s next hop across packet sequence. `recovery_time_s` is populated only for the recovery scenario when a post-recovery delivery exists.

B.3 Aggregate output

Field family	Statistics stored	Caution
PDR	mean, sample SD	n=5 per mode/scenario
Loss ratio	mean, sample SD	Complement of PDR at trial level
Mean latency	mean, sample SD	Delivered packets only
Median / p95 latency	mean across trials	Not pooled packet-level percentile
Throughput	mean, sample SD	Payload rate, not PHY capacity
Route changes	mean, sample SD	Simulator transitions only
Recovery time	mean	NaN outside recovery/censored cases

Table B.3: Aggregate output fields

C Software Test Definition Review

The table below summarizes the Python test definitions. In the recorded validation, the root Python suite passed 9 tests. The simulation/inet/ml-routing suite ran 15 tests: 11 passed and 4 runtime integration tests were skipped because they require an activated OMNeT++/INET environment. Passing software tests establish only the behaviors they exercise; skipped integration cases and physical hardware remain unverified.

Test	Defined check	Does not establish
test_topology_has_alternative_routes_and_six_nodes	Graph has at least six nodes and multiple S-to-G paths	Software topology assertion
test_baseline_delivers_and_packet_has_required_fields	Loss-free baseline reaches gateway and packet history initialized	Simulator behavior
test_link_failure_and_disconnected_gateway_handled	Unavailable source edges yield route_unavailable	Graph state handling
test_loss_hop_limit_and_loop_safeguards	Full loss and hop-limit conditions produce expected statuses	Simulator guard paths
test_seed_reproducibility_and_ml_changes_decisions	Repeat baseline reproducibility; model train and ML run	Synthetic/software behavior
test_dataset_repeatable_and_has_multiple_classes	Fixed-seed dataset equality and two target classes	Synthetic generator behavior
test_duplicate_suppression_and_ml_hysteresis	Duplicate helper and previous-route margin	Python helper/route behavior
test_model_load_failure_and_experiment_csv_exports	Missing/corrupt load and output CSV creation	File/API behavior
test_metrics_and_csv_scenarios	Metric formula and scenario-list expectations	Metric function behavior

Table C.1: Existing test definitions and claim boundary

Test evidence should include the environment and command output, but establishes only software behavior covered by those tests. It does not verify ESP32 radio behavior, on-device model deployment, visual UI quality, or physical measurements. The INET ML project has separate recorded build and simulation outputs; skipped integration tests remain distinct from those completed runs.

D Routing Pseudocode and Source Mapping

D.1 Baseline next-hop selection

```
function minimum_hop_next_hop(graph, current, gateway):
    paths = all shortest paths over available edges
    if no path or path has no next vertex:
        return (None, route_unavailable)
    choose lexicographically smallest shortest path
    return (path[1], minimum_hop)
```

The actual implementation uses NetworkX and returns a path-derived next hop. The diagram and pseudocode describe current code behavior at a high level; they are not embedded firmware code.

D.2 ML candidate selection

```
function ml_next_hop(graph, node, gateway, packet_history, model, load, previous):
    candidates = []
    for neighbor in sorted(graph.neighbors(node)):
        reject if neighbor in packet_history
        reject if link unavailable
        tail = deterministic shortest route from neighbor to gateway
        reject if no tail or tail returns through node
        features = link_features(link, remaining_tail_hops, load)
        q = model probability for success class on one link attempt
        tail_success = product(1 - configured_loss(edge)) along tail
        score = (1 - q) + (1 - tail_success)
        append (score, neighbor)
    if candidates empty:
        return minimum-hop fallback
    best = lowest-score candidate, tie-broken by neighbor ID
    if previous candidate remains within 0.08 cost margin:
        retain previous
    else:
        choose best
```

D.3 Data/metric functions

$$\text{PDR} = \text{delivered} / \text{generated};\ \ \text{loss ratio} = 1 - \text{PDR} \tag{D.1}$$

$$\text{Mean latency} = (\Sigma\ \text{latency}_i \text{ for delivered packets}) / \text{delivered count} \tag{D.2}$$

$$\text{Throughput (kbps)} = (8 \times \text{payload_bytes} \times \text{delivered_count}) / \text{observation_seconds} / 1000 \tag{D.3}$$

The metric implementation returns NaN for latency when no packets are delivered, while retaining packet counts. A consumer must preserve missing values rather than interpret them as zero.

E Simulation Configuration Reference

E.1 Python defaults

Parameter	Default / behavior	Source
Minimum node count	6	SimulationConfig/build_topology
Source / gateway	S / G	SimulationConfig
Packet count	100	SimulationConfig
Rate	5 packets/s	SimulationConfig
Payload	64 bytes	SimulationConfig
Hop limit	12	SimulationConfig
Seed	17	SimulationConfig
Link RSSI	−60 dBm default	Link/build_topology
Link loss	0.05 default	Link/build_topology
Latency	20 ms default	Link/build_topology
Capacity	250 kbps default	Link/build_topology
ML margin	0.08	routing/engines.py

Table E.1: Python simulation parameters

E.2 INET configuration values

Parameter	Configured value	Evidence status
Network	p12ganesh.mesh.IoTMeshNetwork	NED/config exists
Node count	6	Configuration only
Simulation duration	60 s	Configuration only
Seed set	17	Configuration only
PHY bitrate	2 Mbps default	Configuration only
Unit-disk range	230 m; degraded config 190 m	Abstraction, not measured
Source app	128 B every 200 ms; start 2 s; stop 58 s	Configuration only
TTL	8	Configuration only
Normal routing	AODV	Configuration only
Outputs	SCA, VEC, RT under results/	No output files present

Table E.2: INET configuration parameters

Other INET configurations describe competing traffic/interference, higher offered traffic, high latency through a 100 kbps rate, a second flow, relay failure, preferred-path failure, and link recovery. Exact scenario scripts should be retained with each run and checked in the pinned tool environment.

F Trial-Level Result Tables

The next tables are descriptive values transcribed from the 70-row trial CSV. Each row is a paired trial index/seed; the baseline and ML values share that key. They remain simulation-derived and should not be presented as physical results.

F.1 Normal paired trials

Trial	Seed	Base PDR	ML PDR	Base delivered	ML delivered	Base mean ms	ML mean ms
0	17	0.93	0.93	93	93	60.0	60.0
1	18	0.94	0.94	94	94	60.0	60.0
2	19	0.86	0.86	86	86	60.0	60.0
3	20	0.85	0.85	85	85	60.0	60.0
4	21	0.79	0.79	79	79	60.0	60.0

This table preserves five paired simulated trials for the normal scenario. It allows inspection of trial variation hidden by scenario means. A paired difference can be calculated by subtracting the baseline outcome from its same-seed ML outcome; it should not be treated as an independent-sample comparison.

F.2 Degraded paired trials

Trial	Seed	Base PDR	ML PDR	Base delivered	ML delivered	Base mean ms	ML mean ms
0	1017	0.47	0.85	47	85	105.0	60.0
1	1018	0.44	0.82	44	82	105.0	60.0
2	1019	0.51	0.83	51	83	105.0	60.0
3	1020	0.52	0.82	52	82	105.0	60.0
4	1021	0.50	0.84	50	84	105.0	60.0

This table preserves five paired simulated trials for the degraded scenario. It allows inspection of trial variation hidden by scenario means. A paired difference can be calculated by subtracting the baseline outcome from its same-seed ML outcome; it should not be treated as an independent-sample comparison.

F.3 Preferred Link Failure paired trials

Trial	Seed	Base PDR	ML PDR	Base delivered	ML delivered	Base mean ms	ML mean ms
0	2017	0.85	0.85	85	85	60.0	60.0
1	2018	0.87	0.87	87	87	60.0	60.0
2	2019	0.94	0.94	94	94	60.0	60.0
3	2020	0.81	0.81	81	81	60.0	60.0
4	2021	0.84	0.84	84	84	60.0	60.0

This table preserves five paired simulated trials for the preferred link failure scenario. It allows inspection of trial variation hidden by scenario means. A paired difference can be calculated by subtracting the baseline outcome from its same-seed ML outcome; it should not be treated as an independent-sample comparison.

F.4 High Loss paired trials

Trial	Seed	Base PDR	ML PDR	Base delivered	ML delivered	Base mean ms	ML mean ms
0	3017	0.24	0.24	24	24	60.0	60.0
1	3018	0.18	0.18	18	18	60.0	60.0

Trial	Seed	Base PDR	ML PDR	Base delivered	ML delivered	Base mean ms	ML mean ms
2	3019	0.29	0.29	29	29	60.0	60.0
3	3020	0.24	0.24	24	24	60.0	60.0
4	3021	0.30	0.30	30	30	60.0	60.0

This table preserves five paired simulated trials for the high loss scenario. It allows inspection of trial variation hidden by scenario means. A paired difference can be calculated by subtracting the baseline outcome from its same-seed ML outcome; it should not be treated as an independent-sample comparison.

F.5 High Latency paired trials

Trial	Seed	Base PDR	ML PDR	Base delivered	ML delivered	Base mean ms	ML mean ms
0	4017	0.94	0.94	94	94	180.0	180.0
1	4018	0.75	0.75	75	75	180.0	180.0
2	4019	0.83	0.83	83	83	180.0	180.0
3	4020	0.84	0.84	84	84	180.0	180.0
4	4021	0.85	0.85	85	85	180.0	180.0

This table preserves five paired simulated trials for the high latency scenario. It allows inspection of trial variation hidden by scenario means. A paired difference can be calculated by subtracting the baseline outcome from its same-seed ML outcome; it should not be treated as an independent-sample comparison.

F.6 Congestion paired trials

Trial	Seed	Base PDR	ML PDR	Base delivered	ML delivered	Base mean ms	ML mean ms
0	5017	0.78	0.78	78	78	117.0	117.0
1	5018	0.70	0.70	70	70	117.0	117.0
2	5019	0.72	0.72	72	72	117.0	117.0
3	5020	0.73	0.73	73	73	117.0	117.0
4	5021	0.73	0.73	73	73	117.0	117.0

This table preserves five paired simulated trials for the congestion scenario. It allows inspection of trial variation hidden by scenario means. A paired difference can be calculated by subtracting the baseline outcome from its same-seed ML outcome; it should not be treated as an independent-sample comparison.

F.7 Recovery paired trials

Trial	Seed	Base PDR	ML PDR	Base delivered	ML delivered	Base mean ms	ML mean ms
0	6017	0.89	0.85	89	85	60.0	60.0
1	6018	0.86	0.85	86	85	60.0	60.0
2	6019	0.80	0.92	80	92	60.0	60.0
3	6020	0.80	0.86	80	86	60.0	60.0
4	6021	0.81	0.79	81	79	60.0	60.0

This table preserves five paired simulated trials for the recovery scenario. It allows inspection of trial variation hidden by scenario means. A paired difference can be calculated by subtracting the baseline outcome from its same-seed ML outcome; it should not be treated as an independent-sample comparison.

G Hardware Test Protocol and Forms

G.1 Test setup record

Field	Record before trial
Trial ID / date / operator	[fill after test]
Board models/revisions/IDs	[record each node]
Firmware build/hash	[record]
Radio channel and peer configuration	[record]
Node placement and distance	[measurement + sketch/photo]
Supply and instrumentation	[model, settings, calibration]
Traffic rate, payload, duration	[predeclare]
Routing mode and model version	[baseline / ML / hash]
Environmental conditions	[record relevant changes]
Trial start/stop and reset events	[timestamps]

Table G.1: Hardware test record form

G.2 Packet accounting procedure

1. Generate a fixed sequence range and store source start time, packet ID, and payload length. 2. At each relay, log receive ID, previous peer, selected next hop, route reason, hop count, and forwarding outcome. 3. At gateway, log receive ID, receive timestamp, path, and duplicate status. 4. At trial end, reconcile every source ID to a terminal state: received, explicit drop, or missing. 5. Report the denominator and treatment of missing packets.

For latency, use source and gateway timestamp basis explicitly. If clocks are not synchronized, collect a request/ACK round-trip measurement and name it RTT. For packet loss, define whether the numerator is unacknowledged attempts, absent source IDs at gateway, or both. These definitions answer different questions and should not be conflated.

For throughput, divide received application payload bits by a declared observation interval. Decide whether the interval includes warm-up, route discovery, or idle time. For control overhead, count discovery, beacon, ACK, retry, and route-update bytes separately from application bytes. For route switches, define whether a change is counted per node, per flow, or across all packets.

G.3 Link failure/recovery procedure

Use a fixed baseline topology and confirm that both candidate paths initially deliver. Select one relay or link and record the exact intervention time. Keep application workload constant. Record last successful packet over the preferred path, first detection of failure, first alternate route decision, first packet forwarded on the alternate route, and first gateway receipt after recovery. Restore the link and record route choice and delivery separately.

Recovery time should state its endpoints. A detection interval is not the same as routing convergence, and neither is identical to end-to-end application recovery. Repeat outage trials with the same intervention procedure and retain packet traces.

G.4 Energy measurement procedure

If energy is in scope, define supply boundary and sample rate before testing. Record voltage and current at the source for baseline and ML runs, including radio idle and active periods. Use matched payload and duration. Report per-node consumption and total system energy if instrumentation permits. State

calibration, sensor burden, and whether logger overhead is included. Do not derive energy from packet count, RSSI, or simulated hop count.

H Reproducibility and Run Records

H.1 Existing documented commands

```
python train_model.py
python run_experiments.py --trials 5 --packets 100 --seed 17 --nodes 6
pytest -q
```

The test command is listed in repository instructions; it was not executed for this report. Commands are reproduced as documentation, not as claimed evidence of current pass status.

H.2 INET run sequence documented by project

```
inet -u Cmdenv -n "src:$INET_ROOT/src" \
  -f simulations/mesh/omnetpp.ini -c Normal -r 0
python3 tools/export_sca.py results --output outputs/omnet_summary.csv
```

The INET README recommends pinned OMNeT++/INET versions and states that environment setup was not attempted. The command sequence should be executed only after environment installation and verification. Preserve output names by run/config; do not overwrite earlier evidence.

H.3 Run manifest template

Manifest key	Value to record
run_id	Unique stable identifier
created_at	Local time zone and ISO timestamp
source_revision	Commit ID or archive checksum
command	Full invocation and arguments
python/tool versions	Exact interpreter, libraries, OMNeT++/INET as applicable
seed and pairing key	Base seed, trial seed, scenario, mode
input hashes	Dataset/model/config checksums
output hashes	Raw logs and summaries
operator/host	Who ran it and environment
status	Completed, failed, or partial with reason

Table H.1: Recommended run manifest

H.4 Provenance reconciliation actions

- Regenerate the dataset and model into a fresh temporary output directory with an explicit seed.
- Record model JSON seed, dataset hash, model hash, and feature schema together.
- Run paired scenarios with explicit base seed and record the source revision.
- Compare grouped raw means and standard deviations with aggregate output.
- Keep new results separate from existing CSVs until provenance is verified.
- Do not edit old artifacts to make their seed metadata appear consistent.

I Model Porting and Parity Checklist

5. Freeze the exact model and feature order.
6. Export every decision threshold, feature index, comparison direction, and leaf probability.
7. Generate golden vectors spanning each tree branch boundary and representative feature ranges.
8. Run each vector through Python model and embedded evaluator; compare class probabilities within a declared tolerance.
9. Test missing, NaN, out-of-range, and stale feature handling.
10. Measure flash, RAM, stack, worst-case inference time, and energy if inference is active during radio workload.
11. Verify that model output class index is mapped to delivery_success correctly.
12. Keep a baseline fallback for invalid model version or feature packet.
13. Record model hash and inference source hash in every trial manifest.
14. Repeat class-wise evaluation on held-out physical runs before enabling ML mode.

The stored Python inference time is not a substitute for target measurements. A model port can differ because of numeric precision, branch translation, feature scaling, and probability calculation. Parity tests should be part of the embedded implementation evidence before model-driven route choices are evaluated.

J Outstanding Information and Submission Gate

Area	Open item	Required evidence / owner
Academic identity	Student, ID, department, institution, guide, year	Confirm from official project records
Hardware	Board SKU/revision, radio configuration, power source	BOM, schematic, photographs
Firmware	Toolchain, packet bytes, route and cache implementation	Source/build and version record
Physical data	RSSI/RTT/loss/throughput/path logs	Raw trials and manifest
Energy	Current/power measurement boundary	Calibrated instrumentation and raw trace
Python output	Original run revision/seed provenance	Reproduced outputs and hashes
ML	Seed/data/model mismatch and failure recall	Matched artifacts, class-sensitive held-out eval
INET	Build/run and route evidence	SCA/VEC/RT files plus exporter cross-check
Figures	Actual hardware and final trace images	Insert only genuine project artifacts
References	PEARL publication fields	Verify venue/year/pages/DOI or omit
Formatting	Final institution template and captions	Supervisor/department review

Table J.1: Finalization checklist

Final submission gate: no unsupported physical result; no unverified INET claim; all performance values trace to raw artifacts; in-text citations resolve; contents and lists match final page layout; student and institutional details are confirmed; all placeholders are replaced or explicitly accepted by the department.

K Evaluation Protocol and Statistical Interpretation

This appendix states how the existing simulation outputs should be read and how a future controlled replication can avoid ambiguous comparisons. The current raw file contains paired baseline and ML rows for each scenario, trial, and seed. The experimental unit is the scenario-trial pair. The two routing modes share topology, seed, and configured scenario within that pair; they are therefore dependent observations. Reporting 70 rows as 70 independent experiments would overstate the amount of evidence.

K.1 Estimands and denominators

For packet delivery ratio, the estimand is delivered application packets divided by source-generated application packets within one run. A dropped packet remains in the denominator. For latency, the current simulator reports mean latency for delivered packets; it does not assign a latency to lost packets. Consequently, a mode that loses more packets can have a deceptively favorable delivered-only latency. PDR and latency must be interpreted together, and future work may report a deadline-success metric that includes loss.

Throughput is received application payload divided by simulation observation time. Its value depends on the observation interval and offered load; it is not radio bit rate. In a physical study, the interval boundary must be fixed before comparisons. Recovery time, route switches, retransmissions, and control overhead each need separate event definitions. Metrics should be computed from per-packet and per-event logs, then independently reconciled with summary rows.

K.2 Paired analysis plan

For every scenario, compute a trial-level difference $d_i = ML_i - baseline_i$ using matching trial and seed. Report all five differences, their mean, sample standard deviation, and a confidence interval with the small sample size stated. Do not infer population-level significance from five deterministic software trials. A bootstrap interval may be exploratory, but resampling must preserve baseline/ML pairing. A physical validation should predeclare the number of repeated trials and randomize run order to reduce time, battery, temperature, and channel-order effects.

The normal scenario is a useful sanity check: identical PDR does not prove identical routing quality under stressed conditions. The degraded scenario shows a large stored PDR gap, but the model and simulation environment share synthetic constructs. This result supports a hypothesis for further testing, not a claim that an ESP32 mesh gains the same improvement. For modes with identical aggregate metrics, inspect raw paths and route choices before concluding the algorithm had no effect; outcomes may be identical while internal decisions differ.

K.3 Required replication checks

- Verify baseline and ML rows share trial, scenario, seed, packet count, topology, and workload.
- Recompute each aggregate directly from raw rows; check population versus sample standard deviation convention.
- Check missing or duplicate trial keys and ensure every intended pair has both routing modes.
- Confirm stored scenario transformations match the source revision used to generate the CSV.
- Retain a manifest linking source revision, environment, command, seed, model, input, and output hashes.
- Plot raw paired differences so aggregate bars do not conceal trial direction or outliers.
- For physical data, blind analysis mode labels until packet accounting and exclusion rules are applied.

No inferential p-value is reported in this report. With five pairs per condition, interval estimates are unstable and should not be used as a substitute for adequate replication. Engineering relevance also requires a predeclared minimum improvement and a non-inferiority bound for latency, energy, route stability, or other constraints.

L Packet Format and State-Machine Design Notes

The physical packet fields below are an implementation proposal derived from the integration plan. They are not a compiled firmware structure. A real protocol must define byte order, field widths, version negotiation, maximum payload, authentication or integrity requirements, and compatibility behavior before nodes exchange production-like traffic.

L.1 Proposed field semantics

Field	Purpose	Validation rule to implement
version	Wire-format compatibility	Reject unsupported version; do not reinterpret unknown fields
message_type	Data, ACK, route control, or diagnostic	Allow-list type and type-specific payload length
source_id / destination_id	End-to-end identity	Use stable node identifiers; validate destination
previous_hop / next_hop	Immediate forwarding context	Bind sender to received peer; reject invalid next hop
packet_id / sequence	Duplicate detection and accounting	Unique within documented wrap and reboot policy
hop_count / hop_limit	Loop and path-length control	Reject exhausted limit; increment only on forward
timestamp or nonce	Timing or replay context	State clock basis; protect against stale duplicates
payload_length / payload	Application content	Bound length by transport and local buffer sizes
integrity field	Corruption detection	Specify checksum or authenticated mechanism and coverage

Table L.1: Proposed packet fields and validation responsibilities

L.2 Receive and forwarding state machine

```
RECEIVE(frame, peer):
  validate frame length, version, type, and integrity
  if packet_id already in bounded duplicate cache: count duplicate; discard
  if destination is this node: deliver once; send/record acknowledgment
  if hop_limit exhausted or this node appears in path history: drop safely
  refresh local link observation for peer with timestamp and sample count
  build eligible candidate set excluding peer/path-loop candidates
  choose next hop using selected mode; if model unavailable, use baseline
  append forwarding context; decrement hop_limit; transmit
  record decision, outcome, retry count, and monotonic local timestamp
```

The source integration plan mentions a bounded duplicate cache, packet history, hop limit, and loop rejection. Cache capacity and expiry must be chosen from maximum packet rate, network size, and expected packet lifetime. Eviction can permit an old duplicate to circulate, so hop limits remain necessary. Reboot behavior must be defined because volatile caches disappear. A sequence number alone is insufficient if it resets without an epoch or source boot identity.

L.3 Acknowledgment and retry behavior

The project plan discusses delivery acknowledgments and retry-based loss estimation. Firmware design must specify whether an ACK confirms radio reception at the next hop or end-to-end delivery at the gateway. Link ACK and end-to-end ACK answer different questions. Retries require an attempt limit, backoff policy, duplicate suppression, and explicit counters. A routing observation should distinguish first-attempt success from eventual success after retries. Otherwise a seemingly reliable link may hide high airtime and energy cost.

Peer discovery and route maintenance also need a bounded policy. Periodic beacons consume airtime; event-triggered probes can react slowly if no traffic exists. Route freshness should be measured from the last successful observation, not inferred solely from a stored RSSI value. If no candidate meets a freshness threshold, safe behavior is to use a documented fallback or hold the packet until a bounded timeout, then record a drop reason.

M Risk Register and Acceptance Gate

Risk priority below is qualitative and intended to organize remaining engineering work. Likelihood and impact require reassessment after a board, firmware, and radio configuration are chosen. A risk is closed only when the listed evidence is reviewed, not when a mitigation is planned.

Risk	Effect	Control	Closure evidence
Synthetic-to-real distribution shift	Model ranks physical links incorrectly	Collect labeled multi-condition physical observations; retain fallback	Held-out physical run report
Failure class missed by current model	Unsafe candidate appears reliable	Threshold and class-aware evaluation; inspect failure recall	Confusion matrix and calibrated decision rule
Inconsistent artifact provenance	Metrics cannot be reproduced	Run manifest and hashes; regenerate into isolated directory	Independent raw-to-aggregate reconciliation
Loop or duplicate forwarding	Packet amplification and wasted airtime	Path history, bounded cache, hop limit, retry bounds	Adversarial packet and loop-injection records
Clock mismatch	Invalid one-way latency	Synchronize clocks or report RTT with clear endpoints	Clock uncertainty or RTT protocol record
Radio interference and channel changes	Unstable delivery and confounded comparisons	Log channel/environment; repeat randomized paired runs	Raw trials with environment notes
Power measurement burden	Incorrect energy conclusion	Calibrated instrument and declared measurement boundary	Calibration and raw current/voltage trace
Incomplete INET execution	Simulation claims not independently supported	Pin versions and preserve native result files	Successful run plus exported-data audit
Model port mismatch	Embedded route differs from Python	Golden-vector parity test and feature-order freeze	Vector-by-vector parity log
Unspecified hardware identity	Another team cannot reproduce setup	Record board revision, MAC setup, power, geometry	BOM, photos, configuration sheet

Table M.1: Qualitative project risk register

M.1 Stage gates

Gate	Entry condition	Pass evidence	Stop condition
G0: reproducible simulation	Source and output files available	Matched hashes, raw aggregate check, paired keys	Unknown provenance or mismatched row counts
G1: link layer	Boards and peer transport configured	Per-link packet/ACK logs; stable fixed-route test	Unexplained duplicate or forwarding loss
G2: multi-hop baseline	Fixed route and gateway logs operational	Complete packet accounting over both paths	Loop, unbounded retry, or missing denominator
G3: learning data	Feature and label pipeline frozen	Train/held-out split by run; physical labels reviewed	Leakage across packets from same run
G4: ML mode	Baseline stable and model parity verified	Paired tests meet predeclared constraints	Failure recall or safety bound unacceptable
G5: final claim	All selected evidence archived	Reproducible figures/tables and approved limitations	Any metric lacks a traceable source

Table M.2: Proposed staged acceptance gates

A gate can pass only for its stated scope. Passing the Python simulation gate does not imply firmware readiness. Passing a fixed-route radio test does not demonstrate adaptive routing. An ML mode that fails a safety threshold should remain disabled while data and model design are revised.

N Field Log Templates and Data Quality Rules

N.1 Packet event record

Column	Type / example	Definition
run_id	string	Joins event to manifest and configuration
node_id	string	Node that emitted this event
monotonic_time_us	integer	Local monotonic time; clock domain stated
event	enum	generated, received, duplicate, routed, forwarded, retry, dropped, delivered
packet_id	string / tuple	Unique end-to-end packet key
peer_id	string or null	Immediate radio peer
route_mode	enum	minimum-hop or ML-assisted
chosen_next_hop	string or null	Forwarding action
rssi_dbm	number or null	Radio metadata with sampling time
retry_index	integer	Zero-based or one-based convention fixed in schema
drop_reason	enum or null	Explicit terminal cause
feature_vector_version	string	Schema/model compatibility marker

Table N.1: Recommended packet event log schema

N.2 Data quality rules

- Require run_id and packet key on every packet event; reject orphan events.
- Use monotonic timestamps within one node; never compare unrelated clock domains as synchronized wall time.
- Treat missing RSSI or latency as missing values with explicit reason, not numeric zero.
- Record feature observation time so stale values can be excluded or modeled deliberately.
- Keep configuration and model version constant within one trial unless experiment explicitly studies a change.
- Record source-generated packet range so absent gateway IDs can be counted as losses.
- Deduplicate only by documented packet identity and preserve duplicate events for diagnosis.
- Retain raw append-only logs; produce summaries as derived artifacts with script version and hash.

N.3 Minimal trial summary

A trial summary should include mode, scenario, run order, seed or physical placement block, packet count, delivered count, lost count, duplicate count, retries, route changes, mean and percentile latency, payload throughput, control bytes, and drop reasons. If a field is not measured, report it as unavailable; do not substitute a simulator default or infer it from a neighboring field. Every summary statistic should be traceable to raw event IDs.

O Claim-to-Evidence Traceability Matrix

Claim type	Current evidence	Safe wording	Evidence needed for stronger claim
Python routing behavior	Source in routing/engines.py	The simulator implements candidate scoring with fallback	Versioned code and behavior trace for each decision
Synthetic classifier performance	Saved JSON/model and generated dataset	Stored evaluation has high aggregate accuracy but misses failure class	Run-grouped physical held-out set and class-wise recall
Scenario PDR differences	70 raw paired rows and aggregates	Stored Python simulations differ in degraded scenario	Repeated physical trial with fixed denominator
INET scenario support	INI configuration and runner scripts	An INET/AODV configuration is prepared	Pinned successful run and native outputs
ESP-NOW transport	Project design notes and vendor docs	Peer transport is a candidate for the planned testbed	Chosen board, firmware, measured multi-hop forwarding
Energy efficiency	No physical current data	No energy conclusion is supported	Calibrated current/voltage trace under paired workload
Real-world robustness	No physical deployment dataset	Robustness remains unverified	Multiple environments, runs, and independent test set

Table O.1: Claim strength and required evidence

The report uses evidence qualifiers so a reader can distinguish implementation from configuration, generated data, and physical measurements. The traceability principle is simple: every numerical statement should name its artifact and unit; every implementation statement should point to source; every planned statement should remain in future tense; and every absent result should be named as unverified instead of silently omitted.

P Reproduction Checklist for an Independent Reviewer

15. Confirm archive or repository revision and inspect working-tree changes before reproduction.
16. Check interpreter and package versions against the project environment description.
17. Verify dataset, model metadata, and model binary hashes before loading.
18. Read feature names and order from training code and compare with inference code.
19. Run the documented training command into a separate output location; do not overwrite saved artifacts.
20. Run a single scenario first and verify packet and event accounting.
21. Reproduce all five paired trials using the same seed policy as the stored output.
22. Recompute aggregate rows and compare to the stored file with declared numeric tolerance.
23. Inspect confusion matrix with positive/negative class meanings and compute failure recall.
24. If INET is in scope, pin exact OMNeT++/INET versions and preserve SCA, VEC, and event logs.
25. For hardware, record board identity, channel, placement, firmware hash, traffic, power, and trial order.
26. Check every chart against its raw table and verify axis units and sample count.
27. Record any deviation as a new run; do not silently alter historic outputs.

Independent reproduction should produce a short signed record containing date, operator, host, source revision, commands, environment, output hashes, discrepancies, and final status. “Reproduced” means the stated outputs were recreated within stated tolerances from the stated sources. It does not imply that physical behavior or external environments were validated.

References

[1] T. Winter, P. Thubert, A. Brandt, J. Hui, R. Kelsey, P. Levis, K. Pister, R. Struik, J. Vasseur, and R. Alexander, “RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks,” RFC 6550, Internet Engineering Task Force, Mar. 2012, doi: 10.17487/RFC6550.

[2] J.-P. Vasseur, M. Kim, K. Pister, N. Dejean, and D. Barthel, “Routing Metrics Used for Path Calculation in Low-Power and Lossy Networks,” RFC 6551, Internet Engineering Task Force, Mar. 2012, doi: 10.17487/RFC6551.

[3] C. L. Dueñas Santos, A. M. Mezher, J. P. Astudillo León, J. Cárdenas Barrera, E. Castillo Guerra, and J. Meng, “ML-RPL: Machine Learning-Based Routing Protocol for Wireless Smart Grid Networks,” IEEE Access, vol. 11, pp. 57401–57414, 2023, doi: 10.1109/ACCESS.2023.3283208.

[4] S. R. Lalani, B. Safaei, A. M. H. Monazzah, and A. Ejlali, “PEARL: Power and Delay-Aware Learning-based Routing Policy for IoT Applications,” in Proceedings of the 2022 CPSSI International Symposium on Real-Time and Embedded Systems and Technologies (RTEST), 2022, pp. 1–8, doi: 10.1109/RTEST56034.2022.9849862.

[5] B. Langpoklakpam and L. K. Murry, “Review on Machine Learning for Intelligent Routing, Key Requirement and Challenges Towards 6G,” Computer Networks and Communications, vol. 1, no. 2, pp. 214–230, Jul. 2023, doi: 10.37256/cnc.1220233039.

[6] L. Xu, S. Han, W. Fu, Z. Zhu, J. Wu, and X. Zhu, “GNN-DRL-Based Intelligent Routing and Resource Allocation Algorithms for Multi-Layer Wireless Mesh Network,” Sensors, vol. 26, no. 4, art. 1170, Feb. 2026, doi: 10.3390/s26041170.

[7] Espressif Systems, “ESP-NOW,” ESP-IDF Programming Guide, v4.4. Available: https://docs.espressif.com/projects/esp-idf/en/v4.4/esp32/api-reference/network/esp_now.html. Accessed: 1 Nov. 2026.

[8] Espressif Systems, “ESP-NOW,” ESP-IDF Programming Guide, latest ESP32 documentation. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html. Accessed: 1 Nov. 2026.

[9] J. Lloret, J. Tomás, P. L. González Ramírez, M. Jiménez-Piedrahita, and M. Stojmenovic, “M2M group-based AI routing protocol for IoT networks,” Computer Communications, vol. 244, art. 108338, Dec. 2025, doi: 10.1016/j.comcom.2025.108338.

[10] INET Framework, “INET 4.7 User’s Guide,” INET Framework documentation. Available: https://inet.omnetpp.org/inet/docs/4.7/users-guide/index.html. Accessed: 1 Nov. 2026.

[11] INET Framework, “AODV,” INET 4.7 API documentation. Available: https://inet.omnetpp.org/inet/docs/4.7/api/types/inet.routing.aodv.Aodv.html. Accessed: 1 Nov. 2026.

[12] OMNeT++, “Documentation,” OMNeT++ simulation framework. Available: https://omnetpp.org/documentation/. Accessed: 1 Nov. 2026.

[13] scikit-learn Developers, “DecisionTreeClassifier,” scikit-learn API reference. Available: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html. Accessed: 1 Nov. 2026.

[14] NetworkX Developers, “Shortest Paths,” NetworkX documentation. Available: https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html. Accessed: 1 Nov. 2026.

[15] Texas Instruments, “LM2596 SIMPLE SWITCHER® Power Converter 150-kHz 3-A Step-Down Voltage Regulator,” datasheet Rev. G. Available: https://www.ti.com/lit/ds/symlink/lm2596.pdf. Accessed: 1 Nov. 2026.